

DriveZero: End-to-End Driving Beyond Human Demonstrations

AD & Robotics, L3 Team

Xiaomi EV

Abstract

Most end-to-end autonomous-driving systems learn by imitating human driving logs, leaving their learned behavior constrained by the quality and behavioral coverage of the recorded trajectories. This report presents DriveZero, an end-to-end system that learns driving behavior beyond human demonstrations. It decomposes driving into a perception model and an action model, pretrains each in the regime best suited to it, and combines them into one end-to-end planner. The two models call for different learning recipes: perception must understand the world, and benefits from massive and diverse visual data; action must interact with it, and requires closed-loop feedback. On the action side, we introduce DriveRL, a mixed-agent closed-loop reinforcement-learning framework. It converts real driving logs into interactive worlds, where a privileged teacher policy is trained with PPO through closed-loop rollouts. For the perception model, DriveVFM consolidates multiple frozen vision foundation models, including DINOv3, SigLIP2, SAM and Depth Anything V2, into a single backbone from raw images alone, requiring no task-specific annotations. DriveZero then unifies the two: a camera-only planner that distills the frozen DriveRL teacher through its rolled-out trajectories. The goal-conditioned teacher can moreover be queried under augmented driving intents, yielding diverse, goal-consistent supervision that logged data cannot provide. On nuPlan, DriveRL with value-guided test-time action search achieves a mean score of 93.57 across the Val14, Test14-hard, and Test14-random community splits in both non-reactive and reactive modes, exceeding the Log-Replay expert on all three splits. DriveZero achieves state-of-the-art performance on NAVSIMv1, NAVSIMv2 and the closed-loop HUGSIM benchmark without any human trajectory supervision: it reaches 95.3 PDMS on navtest, surpassing the human driver (94.8), 57.1 EPDMS on navhard, and 46.6 HD-Score zero-shot on HUGSIM.

Report: Technical Report, September 2026

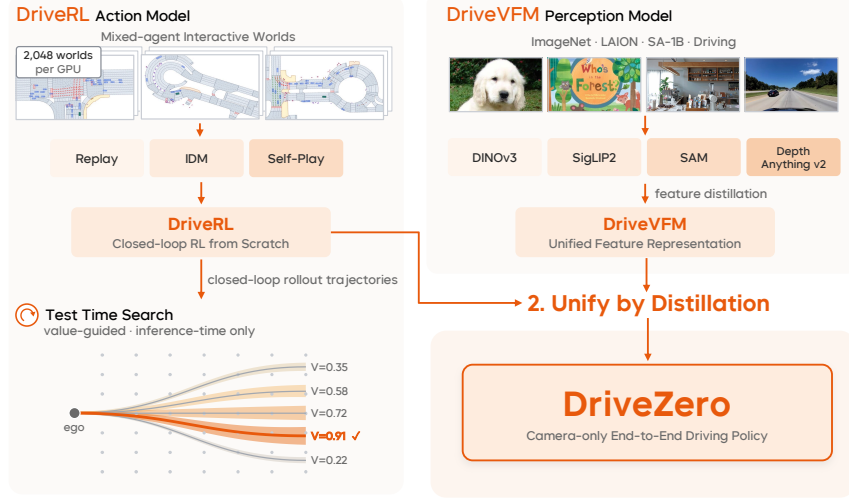
Website: <https://xiaomiauto13.github.io/DriveZero>

1 Introduction

End-to-end autonomous driving aims to map onboard observations and navigation intent directly to vehicle motion [5]. Most existing systems learn this mapping by imitating human driving logs [7, 8, 28, 30, 31, 42, 45]. This paradigm is scalable and stable, but it makes the recorded human trajectory the sole source of supervision, leaving the learned behavior limited by the quality and coverage of the logs. Each logged scene contains only one realized future, even when multiple actions would be valid; safety-critical deviations and recovery maneuvers are rare; and states induced by the learned policy are absent from the offline data. Consequently, errors can compound once the policy leaves the demonstration distribution [61].

Reinforcement Learning (RL) offers a different source of driving behavior. By optimizing explicit objectives through closed-loop interaction, an RL policy can observe the consequences of its own actions, turn failures

1. Decoupled Pretraining of Action and Perception



3. Performance

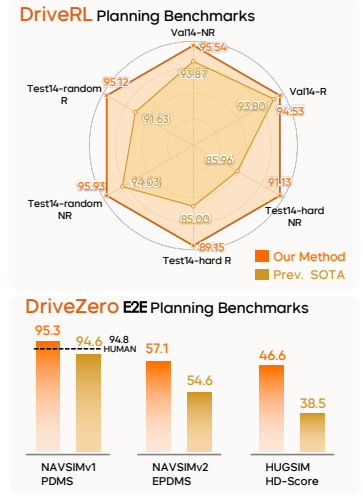


Figure 1 Overview of our end-to-end driving system. (1) Decoupled pretraining. The action model, DriveRL, is a privileged policy trained from scratch with closed-loop RL in mixed-agent interactive worlds built from real nuPlan logs. At inference, a value-guided test-time search rolls out several sampled actions, scores them using short-horizon rewards and the critic, and conditionally replaces the modal action with a higher-value candidate. The perception model, DriveVFM, distills frozen vision foundation models into a single driving backbone from raw images, without task-specific labels. **(2) Unification by distillation.** DriveZero initializes its encoder from DriveVFM and learns from DriveRL rollouts instead of human trajectories, yielding a camera-only end-to-end planner. **(3) Performance.** DriveRL exceeds the log-replay expert and prior methods on all six nuPlan closed-loop settings; DriveZero surpasses the human driver on NAVSIMv1 and sets the state of the art on NAVSIMv2 and HUGSIM without any human trajectory supervision.

into training signal, visit perturbed states, and learn how to recover from them [14, 29, 65]. Thus, RL is not restricted to reproducing the single action sequence chosen by a human driver; it can optimize behavior beyond the demonstrations contained in the logs. Bringing this advantage to end-to-end driving, however, is difficult: direct visual RL couples sample-intensive exploration with costly visual simulation, limiting its scalability [62]. This creates a central tension: closed-loop RL is well suited to learning how to drive, whereas the camera-only policy required for deployment is poorly suited to large-scale online exploration. Privileged RL teachers followed by visual-policy distillation [100] have recently regained attention as a way around this bottleneck [62, 86]. ROACH distills a CARLA RL coach into a monocular policy, while Gigapixel transfers a vector-observation RL teacher to a pixel-based student through self-play Dagger. These methods train the policy in closed loop but take the visual encoder off the shelf. We instead pretrain a perception model and an action model separately, each in the regime best suited to it: perception benefits from large, diverse image collections and rich representation supervision, whereas action benefits from high-throughput closed-loop interaction over compact structured states. We then reunify the two through distillation into a deployable camera-only policy.

For the action model, we introduce **DriveRL**, a mixed-agent closed-loop reinforcement-learning framework. DriveRL converts real nuPlan logs [2] into interactive worlds, where each traffic participant receives an independent behavior provider through a common physical state-action interface, so that log replay, rule-based behaviors, and learned policies coexist within one scene. In these worlds we train the teacher of our system, a privileged policy, with PPO [68] through closed-loop rollouts. The resulting 5.70M-parameter teacher policy observes structured scene state and navigation, outputs bounded Beta distributions over longitudinal jerk and tire steering-angle rate, and controls the vehicle directly without trajectory refinement. PPO also trains a value function (a.k.a critic). We exploit it by a value-guided action search in test-time to cope with out of domain cases for the learned policy. It samples several top actions from the policy, rollouts for a few steps. The final policy is replaced by the best sampled action only if its estimated return exceeds

that of the policy mode by a fixed margin. On nuPlan, DriveRL achieves a mean score of 93.01 across the Val14, Test14-hard, and Test14-random [8, 17] community splits, exceeding the Log-Replay expert in all three sets, and test-time search raises the mean further to 93.57. The learned behavior thus already surpasses the demonstrations that seed its training worlds.

We build the perception model with **DriveVFM**. Rather than coupling a backbone with multiple annotated perception tasks, DriveVFM consolidates frozen vision foundation models (DINOv3 [71], SigLIP2 [78], SAM [35], and Depth Anything V2 [90]) into a single driving backbone from raw images alone, following the agglomerative distillation of RADIO [60]. Since the supervision comes entirely from frozen foundation-model features, training requires no detection, segmentation, lane, or depth labels. This allows the pretraining corpus to freely mix web-scale imagery [35, 63, 67] with driving scenes [2, 72, 89], so the backbone acquires driving-relevant semantics, geometry, and spatial structure at scale. On top of this backbone, **DriveZero** completes the system: a camera-only planner that encodes multi-view images with DriveVFM, decodes multiple trajectory proposals, and learns through winner-takes-all distillation against trajectories rolled out by the frozen DriveRL teacher, with a learned scoring head ranking the proposals at inference. Its training signal thus comes from reinforcement-learned behavior rather than from human demonstrations. Because the teacher is goal-conditioned, the same scene can further be queried under augmented driving intents, yielding diverse, goal-consistent supervision that logged data cannot provide. This augmentation significantly diversifies the training data, increasing the available supervision signals in the training.

DriveZero reaches 94.8 PDMS on NAVSIMv1 navtest without any human trajectory supervision, on par with the human driver (94.8). Scaling its training data with simulation [76] sets the state-of-the-art on all three benchmarks, with 95.3 PDMS on navtest, 57.1 EPDMS on NAVSIMv2 navhard, and 46.6 HD-Score on the closed-loop HUGSIM benchmark. In ablation studies, teacher supervision with goal augmentation surpasses human-trajectory supervision, and an additive study shows that each teacher foundation model contributes cumulatively to DriveVFM. Together, these results validate the decomposition: a reinforcement-learned action model freed from human demonstrations, a perception model pretrained without task annotations, and a distillation that unifies them into a camera-only planner surpassing imitation-based counterparts.

2 Methodology

Our methodology consists of three stages: learning driving behavior, pretraining visual representations, and transferring the learned behavior to a camera-only policy. Section 2.1 describes DriveRL, including its structured inputs and policy network, mixed-agent simulation, and RL objective. Its learned critic further supports value-guided test-time action search. Section 2.2 introduces DriveVFM and its multi-teacher visual representation distillation. Finally, Section 2.3 presents DriveZero, which reunifies the two pretrained models through multimodal trajectory distillation, proposal scoring, and goal-conditioned augmentation.

2.1 DriveRL: Learning a Privileged Teacher from Scratch with Closed-Loop RL

DriveRL is a closed-loop reinforcement learning system that trains a privileged teacher driving policy from scratch, following the recent evidence that large-scale closed-loop RL alone can produce robust driving behavior [14, 29]. At each step, it receives a privileged structured observation together with goal points that express navigation intent and outputs a distribution over actions that control longitudinal jerk and tire steering-angle rate. Closed-loop training requires an interactive world in which the ego vehicle rolls out its own actions and the surrounding traffic participants evolve alongside it, either by replaying their real driving behavior or by following dynamically consistent models that react to the ego. DriveRL builds such worlds from real nuPlan logs using a mixed-agent simulator. Each background actor follows log replay, a rule-based model, or a learned policy, while the simulator runs up to 196,608 worlds in parallel across 96 GPUs. Within these worlds, DriveRL is optimized with PPO [68] against a reward that combines hard safety events, goal arrival, and soft driving-quality terms. It starts from random initialization and receives no imitation pretraining; logged data only seeds the scenes and navigation goals.

2.1.1 The Structured Inputs and Policy Network

At each step t , the structured observation O_t contains the ego vehicle, surrounding traffic participants, the local vector map, and traffic-light states. The ego and each traffic participant are represented by five frames in total, including the current frame and four preceding frames sampled at 5 Hz, up to 96 participants; the map is represented by up to 256 tokens of local vector elements, with traffic-light states attached to the elements they govern. Navigation information is provided by goal points G_t in the ego frame.

Goal construction. The goal points specify the positions the ego vehicle should reach in the near future. During training, DriveRL constructs a two-point goal representation from future ego positions in the log, using either the same position for both points or two distinct positions. At deployment, it selects two goal points, a near and a far anchor, along the current route. Their look-ahead distances scale with the vehicle’s speed, and the anchors are recomputed at every step. Despite their different construction, both training and deployment goals use the same two-point, permutation-invariant representation.

Policy network. Three dedicated encoders embed the agent, ego-kinematics, and map inputs to 256-dimensional tokens. The two ego-frame goal points are embedded using sinusoidal positional encoding [79] followed by a shared MLP, then mean-pooled into a single goal feature. Using the ego token as the only query, two cross-attention layers aggregate agent context, followed by an ego-to-map cross-attention layer over map tokens to extract map context. The resulting ego token, goal and kinematics embeddings, and map context are concatenated and fused by a shared MLP before entering the policy head. The complete policy contains 5.7M parameters.

Beta action distribution. The action head defines independent Beta distributions for normalized jerk and steering-rate commands [10]. Following CaRL [29], we restrict the shape parameters to $(1, \infty)$ by adding 1 to the softplus activation function, which keeps each distribution unimodal and avoids overweighting extreme commands. PPO samples actions during training, while evaluation uses the mode of each distribution. The normalized commands are then mapped to bounded physical commands and executed by a kinematic bicycle model (Appendix A.3).

2.1.2 Mixed-Agent Simulation

Agent behavior composition. The simulator assigns each background actor an independent behavior provider through a common physical state-action interface. The runtime supports three provider categories: log replay, rule-based behaviors such as IDM and front-vehicle braking, and learned policies such as Self-Play. This allows heterogeneous behaviors to coexist in one scene. In the reported training setup, background vehicles use a batched approximation of IDM, a small subset may be controlled by front-vehicle braking, and pedestrians and other non-vehicle actors remain on log replay. The Self-Play provider is described next.

Self-play extension. Inspired by GigaFlow [14], the mixed-agent runtime also supports a learned provider. The policy under training can control selected background vehicles in addition to the ego. Each policy-controlled vehicle receives its own ego-centric observation and goal, and their actions jointly advance the scene, while the remaining actors stay under IDM or log replay. By default, DriveRL controls only the ego vehicle. Self-play is the configuration we use for real-world deployment (Section 3.3). Rule-based or replayed traffic does not respond to the ego the way real drivers do, whereas policy-controlled background vehicles expose DriveRL to more natural interactions during training. On nuPlan the benefit is small. We attribute this to the benchmark itself, whose closed-loop evaluation drives background vehicles by IDM and therefore does not reward more natural interaction behavior. Tables A6 and A7 report the configuration and results.

Scene-consistent actor insertion. During closed-loop RL training, DriveRL initializes each world from a recorded nuPlan scene and background actors enter the scene according to the log. Some actors appear only later in the recording. Inserting them at the logged entry time is consistent with the original log, but once the ego has deviated from the logged trajectory during rollout, an inserted actor may land in an implausible position relative to the ego, or even collide with it. DriveRL therefore gates the insertion with a

scene-consistency check: a late-entering actor is released only when it is present in the current log frame, the ego pose is within 5 m and 0.35 rad of the logged ego pose, and it would not collide with any visible actor. Once released, the actor follows its logged trajectory.

Parallel PPO training. The simulator implements the entire rollout loop as batched GPU operations, including background-agent behaviors, vehicle dynamics, reward computation, and scene editing. DriveRL uses 2,048 worlds per rank. We also find that a larger PPO minibatch improves training performance. Rollout and optimization settings are specified in Section 3.1 and Appendix A.1.

Training objective. DriveRL is optimized using hard-event penalties h_t , a one-time goal-arrival bonus g_t , and six soft driving-quality scores $q_{t,k} \in [0, 1]$ covering safety, compliance, and comfort. Hard events terminate the rollout, whereas goal arrival does not. Let d_t indicate a hard termination and $\mathcal{K}$ denote the six soft criteria. The scalar reward is

$$r_t = h_t + (1 - d_t) \left(g_t + \frac{1}{H} \prod_{k \in \mathcal{K}} q_{t,k} \right), \quad (1)$$

where $H = 110$ is the rollout horizon. The multiplicative term requires the soft criteria to be jointly satisfied. PPO optimizes this scalar reward. The critic is decomposed into one value channel per reward term, with the channels summing to the total return. Full reward and critic details are given in Appendix A.4.

2.1.3 Value-Guided Test-Time Action Search

PPO compresses the behavior discovered during closed-loop training into an action distribution, but modal execution discards both the policy’s local action diversity and the critic’s estimate of delayed return. We use these signals for value-guided action reranking at inference time, following the broader principle of using policy and value estimates to guide test-time search (TTS) [66, 69]. Rather than training a separate planner, the procedure compares alternative first actions using the same ego dynamics, reward definition, and discount factor as PPO, together with a short-horizon approximation of background-actor motion.

Policy-supported action proposals. At each planner update, we retain the mode of the Beta policy as candidate 0 and sample $N - 1$ alternative first actions from the same policy. Retaining the modal action makes the search a conservative extension of the deployed policy. Additional computation changes the decision only when a policy-supported alternative is predicted to be meaningfully better.

Short-horizon closed-loop evaluation. Each candidate is evaluated through a rollout of L transitions. The candidate determines the first ego action, while subsequent actions are given by the mode of the same policy from the candidate-specific states. Candidates therefore differ not only in their initial control but also in the continuation induced by the resulting state. During rollout, background actors are extrapolated using a constant-turn-rate-and-acceleration model with bounded acceleration and yaw rate. The rollout is deliberately short, intended to expose immediate differences in safety, route compliance, and comfort rather than to model long-horizon traffic interaction.

Reward-consistent scoring. For candidate i , let r_ℓ , d_ℓ , and O_ℓ denote the reward, termination indicator, and observation at TTS rollout step ℓ , where r_ℓ and d_ℓ are computed online along the rollout. The candidate is scored by

$$S^{(i)} = \sum_{\ell=0}^{L-1} \gamma^\ell \prod_{u < \ell} (1 - d_u) r_\ell + \gamma^L \prod_{u < L} (1 - d_u) V(O_L), \quad (2)$$

where γ is the discount factor, V is the teacher critic’s estimate of the discounted return. A terminal transition thus contributes its own reward, while all later rewards and the bootstrap are masked. Reusing the training reward, discount factor, and critic keeps candidate ranking aligned with the objective optimized by PPO.

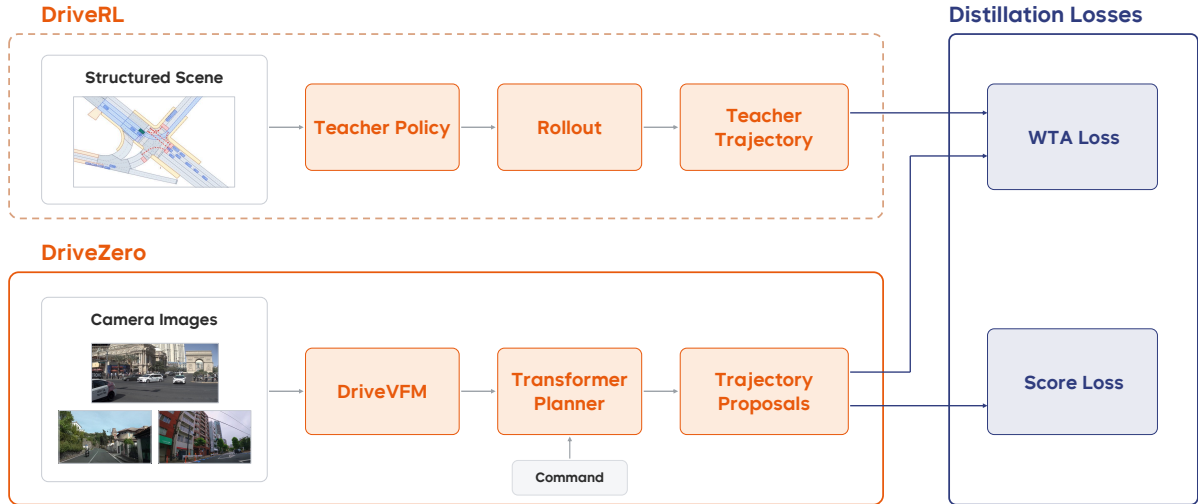


Figure 2 Overview of the end-to-end student policy distillation framework. The privileged DriveRL teacher rolls out a 20-step trajectory from structured scene observations. The camera-only student DriveZero uses DriveVFM features and a command-conditioned Transformer planner to generate multiple 20-step trajectory proposals. Winner-takes-all (WTA) trajectory supervision transfers the teacher behavior, while the proposal-scoring branch is trained against PDM targets. The teacher is used only during training and is removed at inference time.

Conservative action selection. Because both the background-actor rollout and the critic are approximate, a small score difference may be noise rather than a better action. We therefore execute the highest-scoring sampled candidate only if its score exceeds that of the modal action by a margin δ , and execute the modal action otherwise.

Computation. All candidates are rolled out as a single batch, so increasing the number of candidates N primarily enlarges parallel computation, whereas increasing the TTS horizon L adds sequential policy, dynamics, and reward evaluations. The two parameters therefore provide distinct ways to exchange inference-time computation for action-selection quality without retraining or modifying the policy.

2.2 DriveVFM: Distilling Multiple Vision Foundation Models into One Driving Backbone

The teacher policy learns from privileged structured observations, whereas the deployable student must infer the information needed for planning directly from camera images. Transferring the driving capability of the teacher policy therefore first requires a strong visual backbone that recovers driving-relevant semantics, geometry, and spatial structure. Driving-oriented representation learning conventionally trains a shared backbone with multiple auxiliary perception heads, such as detection, lane estimation, segmentation, and depth [28, 75, 83], which depends on large quantities of task-specific annotations. General purpose vision foundation models offer transferable representations without any of this, but no single model captures all driving-relevant capabilities equally well.

DriveVFM replaces manually annotated auxiliary tasks with heterogeneous frozen foundation models, each acting as a learned proxy for a family of perception objectives: DINOv3 [71] provides spatial structure and visual correspondence, SigLIP2 [78] contributes image-level semantics and open-vocabulary information, SAM [35] supplies boundary-sensitive features analogous to segmentation supervision, and Depth Anything V2 [90] offers geometric cues. By directly matching their frozen features, DriveVFM consolidates these complementary capabilities into a single backbone, following the agglomerative distillation of RADIO [60].

Architecture and objective. Given an image I , a shared Vision Transformer [20] produces one summary token per foundation model and a common set of spatial patch tokens. The model-specific summary tokens

let heterogeneous image-level semantics coexist, whereas the shared patch tokens form a unified dense representation for downstream spatial reasoning. Each frozen foundation model is associated with a lightweight adaptor containing separate projections for its summary and patch representations. The supervision is asymmetric, matching each model where its representation is strongest: DINOv3 supervises both summary and patch tokens; SigLIP2 supervises only the summary token; and SAM supervises only the patch tokens. As for Depth Anything, it can enter through the same interface by matching its visual features or distilling its depth predictions. We choose to distill its visual features rather than its depth predictions, as feature distillation performs better in our experiments. Summary features are matched by cosine distance, while spatial patch features are optimized by mean-squared error. For the set of foundation models $\mathcal{T}$, the objective is

$$\mathcal{L}_{\text{DriveVFM}} = \sum_{k \in \mathcal{T}} \left(\lambda_k^{\text{sum}} \mathcal{L}_k^{\text{sum}} + \lambda_k^{\text{patch}} \mathcal{L}_k^{\text{patch}} \right). \quad (3)$$

Balancing heterogeneous foundation models. These frozen models produce features with substantially different scales, variances, and anisotropic activation distributions. A model with larger or more concentrated activations may dominate the gradients and bias the shared DriveVFM backbone toward its feature space. Per-model loss weights cannot fix this, because the imbalance also exists across feature dimensions within a single model. DriveVFM therefore applies PHI Standardization (PHI-S) [59] to each model’s patch features. With mean and covariance estimated offline, PHI-S centers the features, applies a PCA–Hadamard rotation that distributes variance equally across the transformed dimensions, and rescales them by a shared scalar. Every model thus presents a similarly conditioned regression target, so its activation statistics no longer determine its effective importance in the joint objective. Summary features are matched by cosine distance and are already insensitive to magnitude.

Stabilizing backbone optimization. During training, query–key dot products in some attention layers can grow rapidly, producing excessive pre-softmax attention logits and eventually causing loss spikes or divergence. Query–Key Normalization [19, 26] can constrain this behavior by modifying the attention computation. DriveVFM instead adopts a simpler intervention inspired by QK-Clip from Kimi K2 [54].

Pretraining schedule and downstream interface. DriveVFM uses a two-stage progressive-resolution schedule. The first stage operates at 256×256 resolution for 600K optimization iterations with a global batch size of 2,048 images. The second stage increases the resolution to 512×512 and continues for another 200K iterations at the same global batch size. Both stages use AdamW [51]. The training corpus combines general-purpose data from LAION-2B [67], ImageNet-21K [63], and SA-1B [35] with driving images sampled from OpenDV [89], nuPlan [2], and Waymo [72]. This mixture preserves broad visual coverage while exposing the backbone to road geometry, traffic participants, and visual conditions encountered in autonomous driving.

After pretraining, the foundation models and their adaptors are discarded. The DriveVFM backbone encodes each camera with shared weights and provides the visual tokens that DriveZero consumes, serving as the camera-side counterpart of the privileged scene inputs used by the teacher policy.

2.3 DriveZero: Distilling the RL Teacher into a Camera-Only Planner

DriveRL acts on structured scene state, while an end-to-end planner must act on camera images. The driving log bridges these two observation spaces: each frame in the log pairs the multi-view images with the structured state, so a trajectory produced by the teacher from the state can supervise the student on the images. DriveZero builds on this correspondence. It is a camera-only end-to-end planner trained open-loop on logged frames, and the training targets come from rolling out the frozen teacher at each frame. The student never enters an interactive environment and no rendering is required. It inherits the behavior that the RL teacher learned in closed loop, and no human demonstrations are used in its training. Figure 2 illustrates the overall framework.

Architecture. DriveZero takes the multi-view images of the current frame, the ego kinematics, and a navigation command as inputs. The navigation command is derived from the goal point given to the teacher [56],

so student and teacher share one driving intent. From these inputs, DriveZero produces M candidate trajectories, each with a predicted score. DriveVFM, fine-tuned with LoRA [27], encodes each camera image, and the output visual tokens are enriched with 3D position embeddings [50]. Following DrivoR [34], learnable registers [16] then compress these tokens into a small set of scene tokens. The ego kinematics and the command are embedded into one ego token, which is added to M learnable trajectory queries. A trajectory decoder maps these queries to the output trajectories through cross-attention to the scene tokens. A separate scoring decoder then encodes each candidate trajectory as a score query, attends to the visual tokens, and predicts the six components of the PDM score [17]. Trajectory generation and proposal scoring are trained with separate objectives.

Teacher rollouts as supervision. For each logged frame, the frozen teacher receives the structured state and a goal point, and is then rolled out with all background actors following the driving log. The resulting teacher trajectory τ^T is represented for the student as a sequence of ego-relative (x, y, yaw) poses and serves as the regression target of a winner-takes-all objective [25]. Each candidate proposal output by the trajectory decoder is compared with τ^T , and only the closest proposal contributes to the trajectory loss:

$$\mathcal{L}_{\text{traj}} = \min_m \text{dist}(\hat{\tau}_m, \tau^T), \quad (4)$$

where $\hat{\tau}_m$ is the m -th proposal and $\text{dist}(\cdot, \cdot)$ is the L_1 distance averaged over points. This assignment allows different trajectory queries to capture distinct driving modes, leaving the selection among them to the proposal scoring branch.

Proposal scoring. For each proposal, the scoring decoder predicts six driving-quality components covering collision avoidance, drivable-area compliance, progress, time to collision, comfort, and driving-direction compliance. These components are supervised by their corresponding PDM targets [17]. Let $\hat{s}_{mk}$ and s_{mk} denote the predicted logit and target for component k of proposal m . The scoring loss is

$$\mathcal{L}_{\text{score}} = \frac{1}{M} \sum_{m=1}^M \sum_{k=1}^6 \text{BCE}(\sigma(\hat{s}_{mk}), s_{mk}). \quad (5)$$

The components are aggregated according to the benchmark scoring rule. At inference time, DriveZero selects the candidate with the highest predicted aggregate score. Candidate trajectories are detached before entering the scoring branch, separating trajectory generation from proposal ranking. The complete objective combines winner-takes-all trajectory distillation with proposal scoring:

$$\mathcal{L}_{\text{DriveZero}} = \lambda_{\text{traj}} \mathcal{L}_{\text{traj}} + \lambda_{\text{score}} \mathcal{L}_{\text{score}}. \quad (6)$$

Goal augmentation. Human driving logs contain only one realized trajectory under one recorded route intent. DriveRL, however, is goal-conditioned and can be queried with alternative driving intents at the same initial scene state. We exploit this property by augmenting the route intent and its associated navigation command before generating teacher supervision. DriveRL consequently produces diverse yet goal-consistent trajectories beyond the single behavior recorded in the log. The same augmented route is used to evaluate student proposals, keeping the navigation command, teacher trajectory, and proposal scores mutually consistent. Goal augmentation therefore expands each logged scene into multiple valid supervision targets and directly exploits the generative capability of the closed-loop RL teacher.

3 Experiments

We evaluate DriveRL on nuPlan [2] and DriveZero on NAVSIMv1 [18], NAVSIMv2 [3], and HUGSIM [103]. DriveRL is evaluated in closed loop using structured scene states, whereas DriveZero takes camera images and follows the standard sensor-input protocol of each benchmark. DriveVFM is evaluated through the downstream DriveZero planner rather than on a separate benchmark. The following sections describe the evaluation protocols, main results, and ablation studies.

3.1 DriveRL

Training. Each training world is initialized from a recorded nuPlan scene, and PPO generates all training experience through interaction. We draw 922,703 scenes from the nuPlan trainval split, exported at 10 Hz with 41 history and 200 future frames, and the data loader takes every second frame, giving a 5 Hz training rate. DriveRL is trained with 2,048 worlds per rank on 96 GPUs, using 110-step rollouts at 5 Hz over a 1:1 mixture of log-replay and IDM scenes. Each update runs four PPO epochs on the discounted return with $\gamma = 0.99$, and training takes 2,400 updates and approximately 21 hours. Table 1 summarizes the architecture and control configuration. Complete rollout, optimization, and input specifications are in Appendix A.

Table 1 DriveRL training configuration. Core architecture and control settings. Complete implementation details are provided in Appendix.

Component	Teacher configuration
Token width / attention heads	256 / 4
Ego-to-agent / ego-to-map attention	2 / 1 layers
Agent history	5 frames at 5 Hz
Agent / map capacity	96 / 256 tokens
Continuous action	Longitudinal jerk and tire steering-angle rate
Normalized action support	$(0, 1)^2$
Physical action ranges	Jerk $[-8, 5]$ m/s ³ ; steering rate $[-0.8, 0.8]$ rad/s
Vehicle-state constraints	Acceleration cap 4.0 m/s ² ; steering-angle cap $\pi/3$ rad
Training / evaluation action	Beta sample / analytic Beta mode

Evaluation and Benchmarks. We evaluate DriveRL on three nuPlan community splits: Val14 (1,118 scenarios), Test14-hard (272), and Test14-random (261) [2, 8, 17]. Each 15-s scenario is evaluated in closed-loop non-reactive (NR) and reactive (R) modes. Background actors follow their logged trajectories in NR, while eligible vehicles are controlled by IDM in R and the remaining actors stay on log replay. We report the standard nuPlan closed-loop score (CLS), which measures progress, safety, compliance, and comfort on a 0–100 scale. DriveRL and DriveRL-TTS share the same checkpoint. DriveRL executes the mode of its Beta policy, whereas DriveRL-TTS applies the value-guided test-time action search described in Section 2.1.3. The two settings therefore differ only in inference-time action selection and require no retraining.

3.1.1 Closed-Loop Comparison

Table 2 compares DriveRL with published methods. DriveRL obtains an unweighted mean score of 93.01 across the six evaluations and exceeds Log-Replay in every non-reactive and reactive setting, showing that closed-loop RL can learn driving behavior beyond the demonstrations that seed its training worlds. It also exceeds CaRL and GigaFlow, the two prior methods trained without human data, on every setting they report. Applying value-guided test-time action search to the same checkpoint raises the mean score by 0.56 points to 93.57, improving five of the six settings while remaining essentially unchanged on Test14-hard R (89.15 vs. 89.18). Detailed component metrics for DriveRL are reported in Appendix A.5.1, Table A5.

3.1.2 Test-Time Scaling

We next isolate the effect of additional inference computation while keeping the DriveRL checkpoint fixed. All runs use lookahead $L = 5$ and switching margin of $\delta = 0.03$. We vary the number of candidates N from 8 to 64 and evaluate on the six settings of Section 3.1.1.

As shown in Table 3, the mean score generally increases with the candidate budget, reaching 93.57 with 64 candidates. The largest gains occur on the non-reactive Test14 splits: Test14-hard NR improves by 1.16 points, while Test14-random NR improves by 1.43 points. Overall, the scaling trend is consistent with the intended role of test-time search as a local policy-improvement operator. A larger candidate set increases the

Table 2 Closed-Loop Performance on the nuPlan [2] Benchmark. “Human” indicates whether human demonstrations are used as training supervision. DriveRL-TTS applies value-guided test-time action search to the same checkpoint as DriveRL. Best scores are **bolded**; second-best scores are underlined.

Method	Human	Val14 $\uparrow$		Test14-hard $\uparrow$		Test14-random $\uparrow$	
		NR	R	NR	R	NR	R
Log-Replay	$\times$	93.53	80.32	85.96	68.80	94.03	75.86
IDM [77]	$\times$	75.60	77.33	56.15	62.26	70.39	74.42
PDM-Closed [17]	$\checkmark$	92.84	92.12	65.08	75.19	90.05	91.63
UrbanDriver [65]	$\checkmark$	68.57	64.11	50.40	49.95	51.83	67.15
PlanTF [8]	$\checkmark$	84.27	76.95	69.70	61.61	85.62	79.58
PLUTO [7]	$\checkmark$	89.04	80.01	–	–	–	–
Diffusion Planner [101]	$\checkmark$	89.87	82.80	75.99	69.22	89.19	82.93
Flow Planner [73]	$\checkmark$	90.43	83.31	76.47	70.42	89.88	82.93
Pi-DiMT [102]	$\checkmark$	89.66	82.95	77.08	71.02	92.45	87.09
Plan-R1 [74]	$\checkmark$	88.98	87.69	77.45	77.20	91.23	90.04
PlannerRFT [36]	$\checkmark$	89.96	84.46	77.16	72.21	90.76	85.80
CaRL [29]	$\times$	93.87	93.12	–	85	–	–
GigaFlow [14]	$\times$	–	93.80	–	–	–	–
Our Methods							
DriveRL	$\times$	<u>95.16</u>	94.25	<u>89.97</u>	89.18	<u>94.50</u>	<u>95.00</u>
DriveRL-TTS	$\times$	95.54	94.53	91.13	<u>89.15</u>	95.93	95.12

Table 3 Test-time Scaling of a Fixed DriveRL Checkpoint. DriveRL directly executes the Beta mode, whereas DriveRL-TTS evaluates policy-supported action candidates using a five-step rollout and a switching margin. Mean is the unweighted average over the six evaluations, and Δ is measured relative to DriveRL.

Setting	Val14 $\uparrow$		Test14-hard $\uparrow$		Test14-random $\uparrow$		Mean $\uparrow$	Δ $\uparrow$
	NR	R	NR	R	NR	R		
DriveRL	95.16	94.25	89.97	89.18	94.50	95.00	93.01	0.00
DriveRL-TTS ($N = 8$)	95.17	94.49	89.97	89.05	94.92	95.11	93.12	+0.11
DriveRL-TTS ($N = 16$)	95.22	94.38	89.98	88.97	95.29	94.98	93.13	+0.13
DriveRL-TTS ($N = 32$)	95.49	94.43	90.95	88.85	95.83	94.84	93.40	+0.39
DriveRL-TTS ($N = 64$)	95.54	94.53	91.13	89.15	95.93	95.12	93.57	+0.56

chance of finding a first action whose short rollout avoids an unfavorable local consequence while retaining a high-value continuation.

3.2 DriveZero

Training Dataset. For training, we use the `navtrain` split from NAVSIM [18], which is built on nuPlan [33] and contains 100K interactive real-world scenarios. Instead of using logged human driving trajectories, the student is supervised by trajectories accumulated from DriveRL rollouts. We additionally use 237K out-of-distribution (OOD) simulation scenes from SimScale [76] for data scaling, and denote the resulting variant as DriveZero-Scale.

Training Details. DriveZero takes as input the four camera views `CAM_F0`, `CAM_B0`, `CAM_LO`, and `CAM_R0`. The planner comprises 64 trajectory proposals, a 256-dimensional planning representation, and a 4-layer proposal decoder. Following DrivoR [34], camera features are compressed using 16 register tokens per camera. During training, the DriveVFM backbone remains frozen, with only rank-32 Q/V LoRA adapters [27] left trainable. We train DriveZero for 25 epochs with a batch size of 256, using AdamW [51] and an initial

Table 4 Pseudo Closed-Loop Performance on the NAVSIMv1 navtest [18] benchmark. Input: GT = ground-truth symbolic inputs; C = camera; L = LiDAR. “Human” indicates whether human demonstrations are used as training supervision. “-Scale” denotes scaling up the training set with simulation data from SimScale [76]. Best scores are **bolded**; second-best scores are underlined.

Method	Input	Human	NC $\uparrow$	DAC $\uparrow$	TTC $\uparrow$	C $\uparrow$	EP $\uparrow$	PDMS $\uparrow$
PDM-Closed	GT	✓	94.6	99.8	89.9	86.9	99.9	89.1
Human Driver	GT	✓	100	100	100	99.9	87.5	94.8
TransFuser [9]	C & L	✓	97.7	92.8	92.8	100	79.2	84.0
DRAMA [95]	C & L	✓	98.0	93.1	94.8	100	80.1	85.5
VAD-v2 [32]	C & L	✓	98.1	94.8	94.3	100	80.6	86.2
DiffusionDrive [45]	C & L	✓	98.2	96.2	94.7	100	82.2	88.1
Hydra-MDP++ [37]	C & L	✓	98.6	98.6	95.1	100	85.7	91.0
Centaur [70]	C & L	✓	99.5	98.9	98.0	100	85.9	92.6
DriveSuprim [93]	C & L	✓	98.6	98.6	95.5	100	91.3	93.5
UniAD [28]	C	✓	97.8	91.9	92.9	100	78.8	83.4
PARA-Drive [82]	C	✓	97.9	92.4	93.0	99.8	79.3	84.0
Epona [98]	C	✓	97.9	95.1	93.8	<u>99.9</u>	80.4	86.2
OneVL [52]	C	✓	98.5	96.9	95.8	100	82.2	88.4
DriveLaW [84]	C	✓	99.0	97.1	96.7	100	81.3	89.1
AutoVLA [105]	C	✓	98.4	95.6	98.0	<u>99.9</u>	81.9	89.1
DriveVLA-W0 [40]	C	✓	98.7	99.1	95.3	99.3	83.3	90.2
Qwen-Drive-1.0 [104]	C	✓	98.6	98.2	95.9	100	84.8	90.7
ReCogDrive [41]	C	✓	97.9	97.3	94.9	100	87.3	90.8
R2SE [48]	C	✓	99.0	97.9	96.4	100	86.8	91.6
iPad [24]	C	✓	98.6	98.3	94.9	100	88.0	91.7
DriveFine [15]	C	✓	98.8	98.6	96.2	100	86.9	91.8
DrivoR-Scale [34]	C	✓	99.1	<u>99.2</u>	96.9	100	91.6	94.6
Our Methods								
DriveRL	GT	✗	99.8	99.9	99.1	99.0	91.5	95.8
DriveZero	C	✗	99.0	<u>99.2</u>	96.0	100	93.1	<u>94.8</u>
DriveZero-Scale	C	✗	<u>99.2</u>	99.4	<u>97.3</u>	<u>99.9</u>	<u>92.5</u>	95.3

learning rate of 2×10^{-4} decayed to zero via cosine annealing. All main-table DriveZero variants use the DriveVFM ViT-L backbone; results with ViT-S and ViT-B are reported in Appendix B.2.

Benchmarks. We evaluate the camera-only student on three complementary benchmarks. NAVSIMv1 navtest [18], built from nuPlan as a subset of OpenScene, and NAVSIMv2 navhard [3] provide pseudo closed-loop evaluation under standard sensor-input protocols. On navtest, we report the Predictive Driver Model Score (PDMS) and its no-collision, drivable-area compliance, time-to-collision, comfort, and ego-progress components. On navhard, we report the two-stage Extended Predictive Driver Model Score (EPDMS) and its safety, compliance, progress, and planning-quality components; its second stage evaluates robustness to Gaussian-Splatting-based ego-state perturbations. We additionally evaluate zero-shot transfer on HUGSIM [103], a true closed-loop benchmark reconstructed from scenes in KITTI-360 [46], nuScenes [1], PandaSet [85], and Waymo [72]. We report route completion and HD-Score over Easy, Medium, Hard, and Extreme scenarios. Zero-shot means no HUGSIM-specific finetuning is used.

3.2.1 Main Results of the End-to-End Student

Pseudo Closed-Loop Leaderboard: Navtest. As shown in Table 4, the RL-trained DriveRL achieves 95.8 PDMS with 4-s rollouts from ground-truth symbolic inputs, outperforming Human Driver (94.8) and PDM-Closed (89.1). It combines near-human safety (99.8 NC and 99.9 DAC) with substantially higher

Table 5 Pseudo Closed-Loop Performance on the NAVSIMv2 navhard [3] Benchmark. * marks methods using ground-truth symbolic inputs; all others use camera inputs. “Human” indicates whether human demonstrations are used as training supervision, and “S.” indicates per-stage EPDM score. “-Scale” denotes scaling up the training set with simulation data from SimScale [76]. Best scores are **bolded**; second-best scores are underlined.

Method	Human	Stage	NC ↑	DAC ↑	DDC ↑	TLC ↑	EP ↑	TTC ↑	LK ↑	HC ↑	EC ↑	S. ↑	EPDMS ↑
PDM-Closed* [17]	✓	S1 S2	94.4 90.5	98.8 90.6	100 95.4	99.5 98.4	100 100	93.5 86.6	99.3 74.2	87.7 91.9	36.0 29.7	- -	56.6
LTF [9]	✓	S1 S2	96.2 77.8	79.6 70.2	99.1 84.3	99.6 98.1	84.1 85.1	95.1 75.7	94.2 45.4	97.6 95.7	79.1 76.0	- -	25.1
NavFormer [39]	✓	S1 S2	96.2 85.7	92.4 81.0	95.7 83.5	99.6 97.6	83.8 90.1	96.0 82.4	94.7 48.2	96.4 94.9	60.9 48.4	- -	34.1
RAP [21]	✓	S1 S2	97.1 83.2	94.4 83.9	98.8 87.4	99.8 98.0	83.9 86.9	96.9 80.4	94.7 52.3	96.4 95.2	66.2 52.4	- -	39.6
GuideFlow [49]	✓	S1 S2	99.6 91.4	98.0 89.5	99.4 95.2	99.3 98.9	79.7 77.5	99.3 89.6	94.9 52.6	97.1 93.6	58.2 51.0	- -	51.5
SimScale [76]	✓	S1 S2	99.6 94.5	99.1 94.2	99.9 95.8	100.0 99.2	69.6 75.8	99.6 92.8	95.8 60.1	95.6 96.1	28.4 43.2	- -	53.2
DrivoR-Scale [34]	✓	S1 S2	99.1 92.3	98.2 91.6	99.3 97.3	99.8 99.1	75.4 75.7	98.7 90.6	94.9 56.1	97.6 98.4	70.2 44.7	- -	<u>54.6</u>
ZTRS [43]	✗	S1 S2	98.9 91.1	97.6 90.4	100.0 95.8	100.0 99.0	66.7 63.6	98.9 89.8	96.2 60.4	96.7 97.6	44.0 66.1	- -	48.1
GigaPixel [62]	✗	S1 S2	99.4 93.7	95.8 92.9	99.4 96.0	99.8 98.9	68.1 62.6	99.6 90.7	91.8 60.2	97.6 98.2	49.8 58.8	77.8 <u>63.5</u>	50.1
Our Methods													
DriveRL*	✗	S1 S2	99.1 94.4	99.6 92.0	100.0 96.4	99.6 98.1	91.1 89.1	98.7 89.1	99.3 73.8	90.9 88.5	18.2 10.9	84.1 65.4	55.6
DriveZero	✗	S1 S2	98.7 90.5	98.9 89.2	99.9 94.6	99.8 98.6	79.5 81.2	98.7 88.3	95.8 57.2	97.6 94.5	51.6 41.3	84.1 60.8	51.5
DriveZero-Scale	✗	S1 S2	98.7 94.6	99.3 95.6	100.0 97.7	99.3 98.4	76.5 76.2	98.4 92.7	95.3 59.9	97.1 96.1	44.4 44.0	<u>82.3</u> 69.1	57.1

driving efficiency, improving EP from 87.5 to 91.5. High-quality, goal-controllable supervision from DriveRL, together with DriveVFM, enables the camera-only DriveZero to reach 94.8 PDMS without human trajectory supervision, outperforming all prior camera-only and camera–LiDAR fusion methods. Scaling with OOD simulation data from SimScale [76] further improves DriveZero-Scale to 95.3 PDMS, providing direct evidence of the scalability of our framework. DriveZero-Scale also preserves strong safety and driving efficiency, with NC and DAC above 99 and EP at 92.5. To the best of our knowledge, **this is the first end-to-end model to outperform the Human Driver on the NAVSIMv1 navtest leaderboard.**

Pseudo Closed-Loop Leaderboard: Navhard. As shown in Table 5, DriveZero achieves an EPDMS of 51.5, outperforming all methods trained exclusively on the *navtrain* split. It also exceeds GigaPixel [62], another RL-Teacher-based approach, by 1.4 points, even though the student is trained open-loop on logged frames and never enters a simulator. Scaling with SimScale [76] data raises the combined score by 5.6 points to 57.1: the Stage 1 score decreases by 1.8 points, while the more challenging Stage 2 score improves by 8.3 points. This establishes a new state of the art, surpassing both DrivoR-Scale [34], which also uses SimScale data, and PDM-Closed [17], despite the latter using ground-truth symbolic inputs.

True Closed-Loop Leaderboard: HUGSIM. As shown in Table 6, by data scaling, the DriveZero-Scale delivers the strongest zero-shot closed-loop performance. It achieves state-of-the-art HD-Score on the Easy, Medium, and Hard tiers, resulting in an average HD-Score of 46.6. This improves on the previous best result

Table 6 True Closed-Loop Performance on the HUGSIM [103] Benchmark. Zero-shot generalization using the DriveZero model from the NAVSIMv1 evaluation. “Human” indicates whether human demonstrations are used as training supervision. “-Scale” denotes scaling up the training set with simulation data from SimScale [76]. Best scores are **bolded**; second-best scores are underlined.

Method	Human	RC $\uparrow$					HD-Score $\uparrow$				
		E	M	H	X	Avg.	E	M	H	X	Avg.
LTF [9]	✓	67.8	35.1	26.2	40.5	38.9	58.9	18.0	9.8	25.9	23.7
VAD [31]	✓	51.3	31.1	25.3	26.5	31.4	36.3	9.5	8.0	11.5	13.4
UniAD [28]	✓	78.4	60.5	33.6	17.8	45.9	64.9	45.8	<u>20.6</u>	6.6	32.7
DrivoR-Scale [34]	✓	70.4	54.4	28.2	39.2	46.4	64.8	50.3	16.8	25.8	38.1
GigaPixel [62]	✗	78.6	60.7	32.7	35.4	50.1	67.4	<u>51.9</u>	19.1	21.6	38.5
Our Methods											
DriveZero	✗	<u>85.5</u>	63.7	31.7	43.3	<u>53.9</u>	78.5	47.2	16.0	28.7	<u>39.4</u>
DriveZero-Scale	✗	87.5	64.9	34.6	<u>39.8</u>	54.5	81.9	60.5	24.7	<u>27.6</u>	46.6

from GigaPixel [62] by 8.1 points (46.6 vs. 38.5). We attribute this gain to the more plausible and diverse trajectories generated by DriveRL on the OOD SimScale [76] data, which enable the Student to exploit the additional data more effectively.

3.2.2 Ablation Studies

Table 7 summarizes the three controlled studies. All ablations use a ViT-S backbone and the `navtrain` split without SimScale data, and are evaluated on NAVSIMv1 `navtest`. We report the PDMS and five of its components so that changes in the aggregate score can be traced to specific aspects of driving quality.

Table 7 DriveVFM and DriveZero ablations on the NAVSIMv1 navtest [18] benchmark.

Study	Configuration	NC $\uparrow$	DAC $\uparrow$	EP $\uparrow$	TTC $\uparrow$	Comfort $\uparrow$	PDMS $\uparrow$
Visual Backbone	DA3 Small [47]	99.10	98.84	89.68	96.21	99.93	93.31
	DA2 ViT-S [90]	98.57	99.21	91.44	95.13	99.96	93.72
	DINOv2 ViT-S [57]	98.86	99.07	91.63	95.46	99.98	93.88
	DINOv3 ViT-S [71]	98.93	99.01	91.31	95.77	99.97	93.88
	DriveVFM ViT-S	99.17	99.14	91.79	96.29	99.98	94.41
DriveVFM Teachers	DINOv3 + SigLIP2 [71, 78]	98.67	98.98	91.55	95.27	99.98	93.69
	+ SAM [35]	98.79	99.11	91.44	96.06	99.97	94.10
	+DA2 [90] = Full	99.17	99.14	91.79	96.29	99.98	94.41
DriveZero Supervision	Human trajectories	98.76	98.95	91.99	95.30	99.98	93.92
	DriveRL trajectories	98.60	98.81	91.64	95.27	99.94	93.61
	DriveRL + goal aug.	99.17	99.14	91.79	96.29	99.98	94.41

Visual foundation model. The first block of Table 7 investigates whether DriveVFM provides a stronger visual representation for end-to-end driving than existing general-purpose foundation models. We replace DriveVFM with DINOv2 and DINOv3 backbones of comparable capacity while keeping the trajectory head, camera inputs, training data, optimization schedule, and evaluation protocol unchanged. This controlled comparison isolates the contribution of visual pretraining from that of the downstream planning architecture. We report both the overall planning score and its safety, progress, and comfort components to determine whether any advantage of DriveVFM extends beyond a single aggregate metric. DriveVFM achieves a PDMS of 94.41, outperforming the DINOv3 ViT-S baseline at 93.88 by 0.53 points. This margin is notable on an already highly saturated benchmark.

Teacher contribution in DriveVFM. The second block of Table 7 examines whether adding foundation-model teachers with complementary capabilities progressively improves DriveVFM. We begin with DINOv3 and SigLIP2 as the base teacher configuration, then add SAM to introduce object-aware and boundary-sensitive supervision, and finally add Depth Anything V2 to incorporate geometric cues. Each resulting backbone is evaluated through the same downstream DriveZero trajectory-prediction task with all other training and evaluation settings unchanged. Adding SAM improves PDMS from 93.69 to 94.10 (+0.41), while further incorporating Depth Anything V2 raises it to 94.41 (+0.31). Together, the two additional teachers yield a 0.72-point improvement over the DINOv3 + SigLIP2 baseline, demonstrating cumulative gains from complementary supervision.

Trajectory distillation and goal augmentation. The final block of Table 7 studies the central supervision components of DriveZero through three progressively stronger settings: human trajectories, trajectories distilled from DriveRL, and DriveRL trajectories combined with goal augmentation. The first comparison replaces logged human futures with trajectories generated by DriveRL. We then enable goal augmentation on top of DriveRL distillation to measure its additional contribution. Although using DriveRL trajectories alone yields a slightly lower PDMS than human supervision (93.61 versus 93.92), adding goal augmentation raises the score substantially to 94.41, outperforming DriveRL-only supervision by 0.80 points and the human-trajectory baseline by 0.49 points. Goal augmentation is what makes teacher supervision surpass human supervision. It cannot be applied to the human-trajectory baseline, since a changed navigation goal would require a counterfactual human trajectory that logged data do not contain. The ability to generate goal-consistent supervision for arbitrary commands is therefore specific to the teacher.

3.2.3 DriveVFM Feature Representation Visualization

Driving scenes contain long-tail obstacles that may not belong to common semantic categories but can still affect downstream planning. The visual backbone must therefore preserve the distinction between the road surface and such scene elements. We examine this property by comparing the dense patch representations of DINOv3 and DriveVFM in Figure 3.

The second and third columns show PCA visualizations of the two representations. For each model, we use an off-the-shelf Mask2Former segmentation model [6] to identify the ground region and average the corresponding patch features to form a ground prototype. We then compute the cosine similarity between this prototype and every patch feature. The resulting heatmaps appear in the fourth and fifth columns, where blue denotes higher similarity to the ground and red denotes lower similarity.

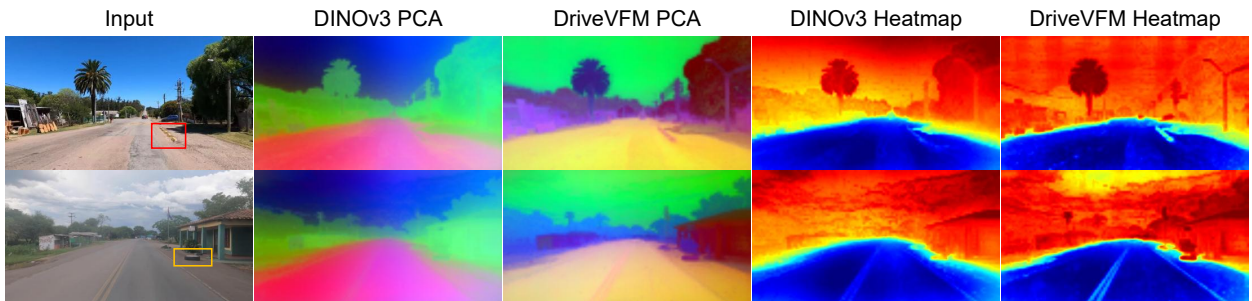


Figure 3 Feature Representation Visualization. From left to right: input images, PCA visualizations of DINOv3 and DriveVFM patch features, and their cosine-similarity heatmaps relative to the ground prototype. Blue indicates higher similarity to the ground and red indicates lower similarity. Boxes mark the long-tail obstacles of interest.

Across both scenes, DriveVFM produces a more consistent response over the road surface and more clearly distinguishes long-tail obstacles and small objects from the ground than DINOv3.

3.2.4 DriveZero Planning Results Visualization

DriveZero-Scale vs. Human Driver. Figure 4 compares the trajectories of DriveZero-Scale and the human driver on `navtest` scenes. In these cases, the human driver hesitates or stays stopped although the road ahead is clear. DriveZero-Scale proceeds through the same scenes with a longer, smoother trajectory while keeping a safe distance from surrounding agents.

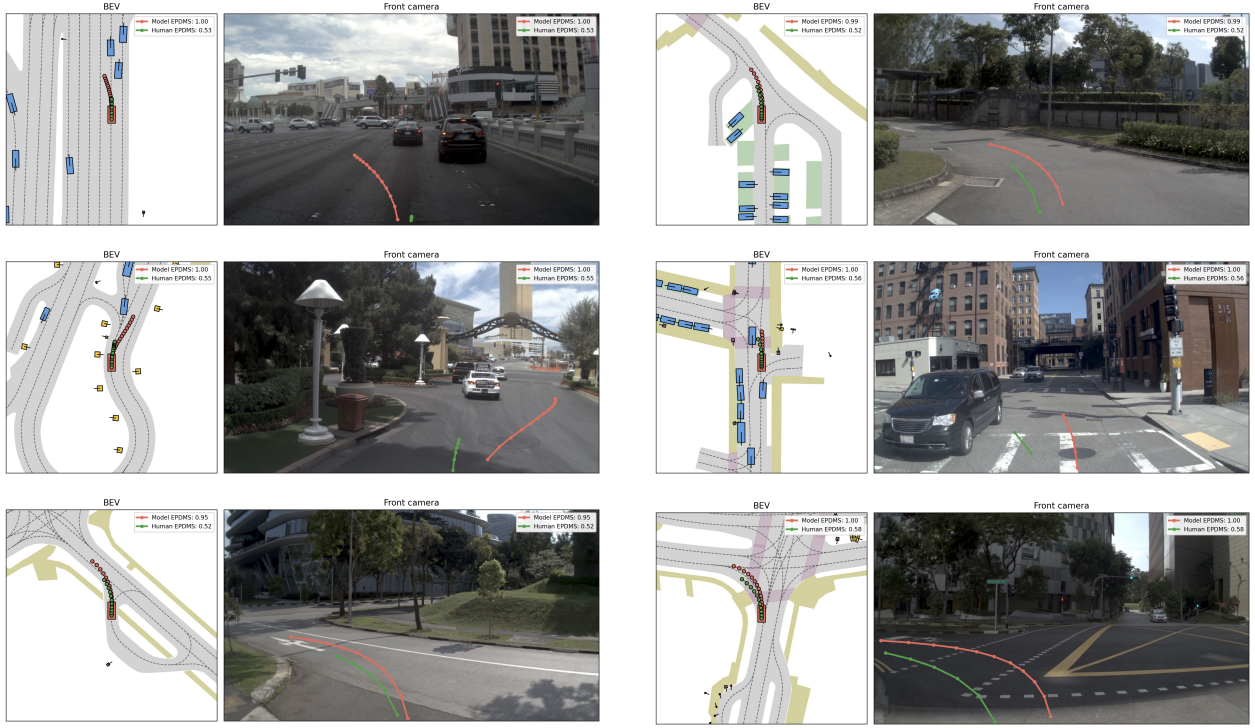


Figure 4 Visualization of **DriveZero-Scale** vs. **Human Driver** in NAVSIM navtest [18] Benchmark.

DriveZero-Scale vs. Previous SOTA Model. Figure 5 compares DriveZero-Scale with DrivoR-Scale, the strongest camera-only baseline in Table 4, on `navtest` scenes. In these scenes, the trajectory of DrivoR-Scale collides with a surrounding agent or leaves the drivable area, whereas DriveZero-Scale stays collision-free and within the road boundary. Figure 6 shows two closed-loop rollouts from HUGSIM. In the first scene, the lead vehicle brakes suddenly. DrivoR-Scale keeps its speed and collides with it, whereas DriveZero-Scale decelerates in time and maintains a safe gap. In the second scene, an oncoming vehicle passes close to the ego lane. DrivoR-Scale swerves away from it and drifts onto the curb, while DriveZero-Scale stays in its lane with only a slight lateral adjustment. More closed-loop rollouts are available on our project page.

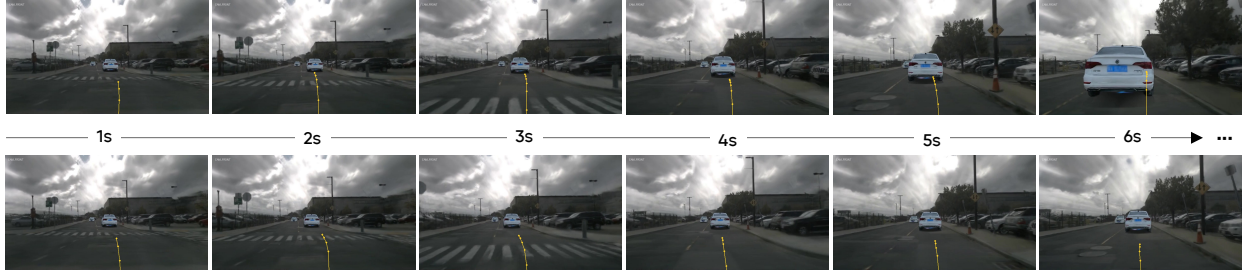
3.3 Real-World Application

We have deployed DriveRL on a small fleet of vehicles and validated it in real traffic. For deployment, DriveRL is trained on our own driving logs with self-play enabled. As on nuPlan, the logs only seed the scenes and goals, and no human demonstration is used for training. The structured state is built from auto-labeled perception ground truth, with recorded onboard perception outputs mixed in as degraded inputs, and extensive domain randomization is applied during training. On the vehicle, the object detection, online mapping and navigation modules provide the surrounding agents, the local vector map, and the goal points, respectively. We demonstrated the system on a crowded urban road with dense traffic, where DriveRL controlled the vehicle in closed loop. **A demonstration video is available on our project page.**



Figure 5 Visualization of **DriveZero-Scale** vs. **Previous SOTA Model** in NAVSIM navtest [18] Benchmark.

DrivoR-Scale



DriveZero-Scale

DrivoR-Scale



DriveZero-Scale

Figure 6 Visualization of **DriveZero-Scale** vs. **Previous SOTA Model** in the HUGSIM [103] Benchmark.

4 Related Work

Closed-Loop Reinforcement Learning for Driving. Closed-loop reinforcement learning allows driving policies to learn from states induced by their own actions rather than only from fixed expert trajectories. Urban Driver [65] trains policies in log-initialized urban scenes using policy gradients, while BC-SAC [53] combines imitation with RL on real logs to improve robustness in safety-critical scenarios. Later work scales model-free training: CarPlanner [97] makes large-scale RL tractable through consistent auto-regressive roll-outs, CaRL [29] shows that scalable simulation and simple rewards suffice for effective policy optimization, and PlannerRFT [36] fine-tunes an imitation-pretrained diffusion planner by efficient closed-loop reinforce-

ment learning. To reduce interaction cost, Think2Drive [38], AdaWM [80], and Raw2Drive [92] learn and explore within world models, extending model-based RL from latent planning to end-to-end driving. Finally, multi-agent training [99] replaces fixed background traffic with learning agents: self-play at scale yields reliable sim agents [12] and robust, naturalistic driving without human data [14], while SPICED [13] shows that a small amount of human driving can regularize self-play toward human-compatible conventions.

Visual Foundation Models for Driving. Camera-based perception and end-to-end driving obtain visual knowledge from both general-purpose and driving-specific pretraining. Many systems inherit backbones trained with ImageNet supervision, self-supervised DINO objectives, or vision–language alignment [4, 57, 63, 71, 78, 96]. Driving-specific pretraining has further explored self-supervised objectives tailored to driving sensor data [64, 88, 91], as well as geometry-oriented pretraining that injects 3D structure into visual representations [58, 87]. While these methods design specialized pretraining objectives for driving, a complementary direction is to reuse and consolidate knowledge already encoded by existing foundation models. RADIO consolidates representations from multiple frozen visual foundation models into a single general-purpose backbone through agglomerative distillation, with PHI-S providing feature standardization across heterogeneous teachers [59, 60]. Following this recipe, we develop DriveVFM, a visual foundation model tailored to autonomous driving. DriveVFM distills DINOv3, SigLIP2, SAM, and Depth Anything into a shared backbone using their features as supervision [35, 71, 78, 90]. This removes the need for task-specific perception labels and allows pretraining to freely combine web images with driving scenes.

End-to-End Autonomous Driving. Most camera-based end-to-end driving policies are trained through behavioral cloning, spanning conditional control, unified perception–planning architectures, and multimodal trajectory generation and selection [11, 23, 28, 30, 31, 42, 44, 45, 81]. Although these methods differ in how trajectories are represented and decoded, their behavioral supervision largely remains the ego trajectory recorded in human driving logs. Recent work instead transfers behavior learned through interactive simulation to visual policies. ROACH distills a CARLA RL coach into a monocular policy [100]; TerraTransfer aligns a visual encoder with a vectorized self-play policy without logged-trajectory targets [86]; and GigaPixel trains a pixel-based student through self-play DAgger in a simplified renderer [62]. Pictura further enables self-play directly from perspective images through high-throughput rendering [94]. Other approaches place visual policies inside photorealistic reconstructed worlds for reinforcement learning, synthetic-data training, or post-training [22, 39, 55, 76]. DriveZero follows the privileged-to-visual transfer paradigm and independently pretrains the action model and visual representation in their respective learning regimes. DriveRL learns goal-conditioned behavior through closed-loop interaction in structured, log-grounded worlds, while DriveVFM learns visual representations from real images. The two are reunited through trajectory distillation, without placing the camera student inside a rendered closed-loop training environment.

5 Conclusion

This report presented an end-to-end driving system whose behavior is not bounded by human demonstrations. Driving is decomposed into a perception model and an action model, each pretrained in the regime that suits it: DriveVFM learns visual representations from raw images by distilling frozen vision foundation models, and DriveRL learns driving behavior from scratch through closed-loop reinforcement learning in log-initialized interactive worlds. DriveZero reunites the two by distilling the frozen teacher’s rollouts into a camera-only planner. On nuPlan, DriveRL exceeds the Log-Replay expert on all three community splits in both non-reactive and reactive modes, showing that closed-loop RL can learn behavior beyond the logs that seed its training worlds. DriveZero achieves state-of-the-art performance on NAVSIMv1, NAVSIMv2, and the closed-loop HUGSIM benchmark without any human trajectory supervision. We believe this recipe, pretraining perception and action separately and unifying them through distillation, offers a scalable path toward end-to-end driving beyond human demonstrations.

6 Contributors

DriveRL: Hao He, Chengcheng Hu, Zirun Su, Heng Zhang

DriveVFM: Haisong Liu

DriveZero: Haisong Liu*, Jinke Li*, Haochen Tian*, Zhenwei Shen, Hongyang Li

Real-World Deployment: Hao He, Zhichao Li, Yunchen Yang, Bochao Huang, Siyu Zhang, Kuangye Chen, Heng Zhang, Xiongjie Zhang, Wentao Dai, Hengchen Dai, Siyuan Liu

Project Lead: Hao He, Zhichao Li, Zehao Huang, Naiyan Wang

*Equal contribution.

References

- [1] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *CVPR*, 2020.
- [2] Holger Caesar, Jurař Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. In *CVPR Workshops*, 2021.
- [3] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, Xunjiang Gu, Caojun Wang, Yakov Miron, Marco Aiello, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Pseudo-simulation for autonomous driving. In *CoRL*, 2025.
- [4] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021. doi: 10.1109/ICCV48922.2021.00951.
- [5] Li Chen, Penghao Wu, Kashyap Chitta, Bernhard Jaeger, Andreas Geiger, and Hongyang Li. End-to-end autonomous driving: Challenges and frontiers. *TPAMI*, 2024.
- [6] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *CVPR*, 2022.
- [7] Jie Cheng, Yingbing Chen, and Qifeng Chen. PLUTO: Pushing the limit of imitation learning-based planning for autonomous driving. *RA-L*, 2024.
- [8] Jie Cheng, Yingbing Chen, Xiaodong Mei, Bowen Yang, Bo Li, and Ming Liu. Rethinking imitation-based planner for autonomous driving. In *ICRA*, 2024.
- [9] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *TPAMI*, 2022.
- [10] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. In *ICML*, 2017.
- [11] Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *ICRA*, 2018.
- [12] Daphne Cornelisse, Aarav Pandya, Kevin Joseph, Joseph Suárez, and Eugene Vinitsky. Building reliable sim driving agents by scaling self-play. *arXiv preprint arXiv:2502.14706*, 2025.
- [13] Daphne Cornelisse, Julian Hunt, Zixu Zhang, Wael Doulazmi, Kevin Joseph, Jaime Fernández Fisac, and Eugene Vinitsky. Human-like autonomy emerges from self-play and a pinch of human data. *arXiv preprint arXiv:2606.19370*, 2026.

- [14] Marco Cusumano-Towner, David Hafner, Alex Hertzberg, Brody Huval, Aleksei Petrenko, Eugene Vinitzky, Erik Wijmans, Taylor Killian, Stuart Bowers, Ozan Sener, et al. Robust autonomy emerges from self-play. In *ICML*, 2025.
- [15] Chenxu Dang, Sining Ang, Yongkang Li, Haochen Tian, Jie Wang, Guang Li, Hangjun Ye, Jie Ma, Long Chen, and Yan Wang. Drivefine: Refining-augmented masked diffusion vla for precise and robust driving. In *ECCV*, 2026.
- [16] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers. In *ICLR*, 2024.
- [17] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *CoRL*, 2023.
- [18] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *NeurIPS*, 2024.
- [19] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *ICML*, 2023.
- [20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. doi: 10.48550/arXiv.2010.11929.
- [21] Lan Feng, Yang Gao, Eloi Zablocki, Quanyi Li, Wuyang Li, Sichao Liu, Matthieu Cord, and Alexandre Alahi. Rap: 3d rasterization augmented end-to-end planning. *arXiv preprint arXiv:2510.04333*, 2025.
- [22] Hao Gao, Shaoyu Chen, Bo Jiang, Bencheng Liao, Yiang Shi, Xiaoyang Guo, Yuechuan Pu, Haoran Yin, Xiangyu Li, Xinbang Zhang, Ying Zhang, Wenyu Liu, Qian Zhang, and Xinggang Wang. Rad: Training an end-to-end driving policy via large-scale 3dgs-based reinforcement learning. In *NeurIPS*, 2025.
- [23] Yinfeng Gao, Deqing Liu, Qichao Zhang, Yupeng Zheng, Haochen Tian, Guang Li, Hangjun Ye, Long Chen, Da-Wei Ding, and Dongbin Zhao. Learning from mistakes: Post-training for driving vla with takeover data. *arXiv preprint arXiv:2603.14972*, 2026.
- [24] Ke Guo, Haochen Liu, Xiaojun Wu, Jia Pan, and Chen Lv. ipad: Iterative proposal-centric end-to-end autonomous driving. *arXiv preprint arXiv:2505.15111*, 2025.
- [25] Abner Guzmán-Rivera, Dhruv Batra, and Pushmeet Kohli. Multiple choice learning: Learning to produce multiple structured outputs. In *NeurIPS*, 2012.
- [26] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *EMNLP*, 2020.
- [27] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [28] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, et al. Planning-oriented autonomous driving. In *CVPR*, 2023.
- [29] Bernhard Jaeger, Daniel Dauner, Jens Beifwenger, Simon Gerstenecker, Kashyap Chitta, and Andreas Geiger. Carl: Learning scalable planning policies with simple rewards. In *CoRL*, 2025.
- [30] Xiaosong Jia, Junqi You, Zhiyuan Zhang, and Junchi Yan. Drivetransformer: Unified transformer for scalable end-to-end autonomous driving. In *ICLR*, 2025.
- [31] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. Vad: Vectorized scene representation for efficient autonomous driving. In *ICCV*, 2023.
- [32] Bo Jiang, Shaoyu Chen, Hao Gao, Bencheng Liao, Qian Zhang, Wenyu Liu, and Xinggang Wang. VADv2: End-to-end autonomous driving via probabilistic planning. In *ICLR*, 2026.

- [33] Napat Karnchanachari, Dimitris Geromichalos, Kok Seang Tan, Nanxiang Li, Christopher Eriksen, Shakiba Yaghoubi, Noushin Mehdipour, Gianmarco Bernasconi, Whye Kit Fong, Yiluan Guo, et al. Towards learning-based planning: The nuplan benchmark for real-world autonomous driving. In *ICRA*, 2024.
- [34] Ellington Kirby, Alexandre Boulch, Yihong Xu, Yuan Yin, Gilles Puy, Éloi Zablocki, Andrei Bursuc, Spyros Gidaris, Renaud Marlet, Florent Bartoccioni, Anh-Quan Cao, Nermin Samet, Tuan-Hung Vu, and Matthieu Cord. Driving on registers. In *CVPR*, 2026.
- [35] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, et al. Segment anything. In *ICCV*, 2023. doi: 10.1109/ICCV51070.2023.00371.
- [36] Hongchen Li, Tianyu Li, Jiazhi Yang, Haochen Tian, Caojun Wang, Lei Shi, Mingyang Shang, Zengrong Lin, Gaoqiang Wu, Zhihui Hao, et al. Plannerrft: Reinforcing diffusion planners through closed-loop and sample-efficient fine-tuning. *arXiv preprint arXiv:2601.12901*, 2026.
- [37] Kailin Li, Zhenxin Li, Shiyi Lan, Yuan Xie, Zhizhong Zhang, Jiayi Liu, Zuxuan Wu, Zhiding Yu, and Jose M Alvarez. Hydra-mdp++: Advancing end-to-end driving via expert-guided hydra-distillation. *arXiv preprint arXiv:2503.12820*, 2025.
- [38] Qifeng Li, Xiaosong Jia, Shaobo Wang, and Junchi Yan. Think2drive: Efficient reinforcement learning by thinking with latent world model for autonomous driving (in carla-v2). In *ECCV*, 2024.
- [39] Tianyu Li, Li Chen, Caojun Wang, Haochen Liu, Kashyap Chitta, Zhenjie Yang, Yuhang Lu, Naisheng Ye, Yihang Qiu, Haochen Tian, Yufei Wang, et al. World engine: Towards the era of post-training for autonomous driving. *arXiv preprint arXiv:2606.19836*, 2026.
- [40] Yingyan Li, Shuyao Shang, Weisong Liu, Bing Zhan, Haochen Wang, Yuqi Wang, Yuntao Chen, Xiaoman Wang, Yasong An, Chufeng Tang, et al. Drivevla-w0: World models amplify data scaling law in autonomous driving. *arXiv preprint arXiv:2510.12796*, 2025.
- [41] Yongkang Li, Kaixin Xiong, Xiangyu Guo, Fang Li, Sixu Yan, Gangwei Xu, Lijun Zhou, Long Chen, Haiyang Sun, Bing Wang, et al. ReCogDrive: A reinforced cognitive framework for end-to-end autonomous driving. In *ICLR*, 2026.
- [42] Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, Yishen Ji, Zhiqi Li, Ziyue Zhu, Jan Kautz, Zuxuan Wu, et al. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024.
- [43] Zhenxin Li, Wenhao Yao, Zi Wang, Xinglong Sun, Jingde Chen, Nadine Chang, Maying Shen, Jingyu Song, Zuxuan Wu, Shiyi Lan, et al. Ztrs: Zero-imitation end-to-end autonomous driving with trajectory scoring. *arXiv preprint arXiv:2510.24108*, 2025.
- [44] Zhenxin Li, Wenhao Yao, Zi Wang, Xinglong Sun, Joshua Chen, Nadine Chang, Maying Shen, Zuxuan Wu, Shiyi Lan, and Jose M Alvarez. Generalized trajectory scoring for end-to-end multimodal planning. *arXiv preprint arXiv:2506.06664*, 2025.
- [45] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, et al. DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving. In *CVPR*, 2025.
- [46] Yiyi Liao, Jun Xie, and Andreas Geiger. KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2d and 3d. *TPAMI*, 2023.
- [47] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.
- [48] Haochen Liu, Tianyu Li, Haohan Yang, Li Chen, Caojun Wang, Ke Guo, Haochen Tian, Hongchen Li, Hongyang Li, and Chen Lv. Reinforced refinement with self-aware expansion for end-to-end autonomous driving. *arXiv preprint arXiv:2506.09800*, 2025.
- [49] Lin Liu, Caiyan Jia, Guanyi Yu, Ziyang Song, JunQiao Li, Feiyang Jia, Peiliang Wu, Xiaoshuai Hao, and Yadan Luo. Guideflow: Constraint-guided flow matching for planning in end-to-end autonomous driving. In *CVPR*, 2026.

- [50] Yingfei Liu, Tianyu Wang, Xiangyu Zhang, Jian Zhang, Wenyu Chang, Yichen Ding, Yichen Wang, Qi Zhao, Jiakui Chen, and Yu Qiao. Petr: Position embedding transformation for multi-view 3d object detection. In *ECCV*, 2022.
- [51] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [52] Jinghui Lu, Jiayi Guan, Zhijian Huang, Jinlong Li, Guang Li, Lingdong Kong, Yingyan Li, Han Wang, Shaoqing Xu, Yuechen Luo, et al. Onevl: One-step latent reasoning and planning with vision-language explanation. *arXiv preprint arXiv:2604.18486*, 2026.
- [53] Yiren Lu, Justin Fu, George Tucker, Xinlei Pan, Eli Bronstein, Rebecca Roelofs, Benjamin Sapp, Brandyn White, Aleksandra Faust, Shimon Whiteson, Dragomir Anguelov, and Sergey Levine. Imitation is not enough: Robustifying imitation with reinforcement learning for challenging driving scenarios. In *IROS*, 2023.
- [54] Moonshot AI. Kimi K2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [55] Chaojun Ni, Guosheng Zhao, Xiaofeng Wang, Zheng Zhu, Wenkang Qin, Xinze Chen, Guanghong Jia, Guan Huang, and Wenjun Mei. Recondreamer-rl: Enhancing reinforcement learning via diffusion-based scene reconstruction. *arXiv preprint arXiv:2508.08170*, 2025.
- [56] OpenScene Contributors. OpenScene: The largest up-to-date 3d occupancy prediction benchmark in autonomous driving. <https://github.com/OpenDriveLab/OpenScene>, 2023.
- [57] Maxime Oquab, Timothée Darques, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [58] Dennis Park, Rares Ambrus, Vitor Guizilini, Jie Li, and Adrien Gaidon. Is pseudo-lidar needed for monocular 3d object detection? In *ICCV*, 2021.
- [59] Mike Ranzinger, Jon Barker, Greg Heinrich, Pavlo Molchanov, Bryan Catanzaro, and Andrew Tao. Phi-s: Distribution balancing for label-free multi-teacher distillation. *arXiv preprint arXiv:2410.01680*, 2024.
- [60] Mike Ranzinger et al. Am-radio: Agglomerative vision foundation model – reduce all domains into one. In *CVPR*, 2024. doi: 10.1109/CVPR52733.2024.01187.
- [61] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.
- [62] Luke Rowe, Roger Girgis, Rodrigue de Schaetzen, Daphne Cornelisse, Alaap Grandhi, Felix Heide, Eugene Vinit-sky, Christopher Pal, and Liam Paull. Scaling self-play for end-to-end driving. *arXiv preprint arXiv:2606.19641*, 2026.
- [63] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *IJCV*, 2015.
- [64] Corentin Sautier, Gilles Puy, Alexandre Boulch, Renaud Marlet, and Vincent Lepetit. Bevcontrast: Self-supervision in bev space for automotive lidar point clouds. In *3DV*, 2024. doi: 10.1109/3DV62453.2024.00017.
- [65] Oliver Scheel, Luca Bergamini, Maciej Wolczyk, Błażej Osiński, and Peter Ondruska. Urban driver: Learning to drive from real-world demonstrations using policy gradients. In *CoRL*, 2022.
- [66] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 2020.
- [67] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. LAION-5B: An open large-scale dataset for training next generation image-text models. In *NeurIPS*, 2022.
- [68] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- [69] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 2017.
- [70] Chonghao Sima, Kashyap Chitta, Zhiding Yu, Shiyi Lan, Ping Luo, Andreas Geiger, Hongyang Li, and Jose M Alvarez. Centaur: Robust end-to-end autonomous driving with test-time training. *arXiv preprint arXiv:2503.11650*, 2025.
- [71] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- [72] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, Vijay Vasudevan, Wei Han, Jiquan Ngiam, Hang Zhao, Aleksei Timofeev, Scott Ettinger, Maxim Krivokon, Amy Gao, Aditya Joshi, Yu Zhang, Jonathon Shlens, Zhifeng Chen, and Dragomir Anguelov. Scalability in perception for autonomous driving: Waymo open dataset. In *CVPR*, 2020.
- [73] Tianyi Tan, Yinan Zheng, Ruiming Liang, Zexu Wang, Kexin Zheng, Jinliang Zheng, Jianxiong Li, Xianyuan Zhan, and Jingjing Liu. Flow matching-based autonomous driving planning with advanced interactive behavior modeling. In *NeurIPS*, 2025.
- [74] Xiaolong Tang, Meina Kan, Shiguang Shan, and Xilin Chen. Plan-R1: Safe and feasible trajectory planning as language modeling. In *ICLR*, 2026.
- [75] Marvin Teichmann, Michael Weber, Marius Zoellner, Roberto Cipolla, and Raquel Urtasun. Multinet: Real-time joint semantic reasoning for autonomous driving. In *IV*, 2018.
- [76] Haochen Tian, Tianyu Li, Haochen Liu, Jiazhi Yang, Yihang Qiu, Guang Li, Junli Wang, Yinfeng Gao, Zhang Zhang, Liang Wang, Hangjun Ye, Long Chen, and Hongyang Li. Simscale: Learning to drive via real-world simulation at scale. In *CVPR*, 2026.
- [77] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical review E*, 2000.
- [78] Michael Tschannen et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.
- [79] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [80] Hang Wang, Xin Ye, Feng Tao, Chenbin Pan, Abhirup Mallik, Burhaneddin Yaman, Liu Ren, and Junshan Zhang. Adawm: Adaptive world model based planning for autonomous driving. In *ICLR*, 2025.
- [81] Junli Wang, Zhihua Hua, Xueyi Liu, Zebin Xing, Haochen Tian, Kun Ma, Hangjun Ye, Guang Chen, Long Chen, and Qichao Zhang. Beyond imitation: Learning safe end-to-end autonomous driving from hard negatives. *ECCV*, 2026.
- [82] Xinshuo Weng, Boris Ivanovic, Yan Wang, Yue Wang, and Marco Pavone. Para-drive: Parallelized architecture for real-time autonomous driving. In *CVPR*, 2024.
- [83] Dong Wu, Man-Wen Liao, Wei-Tian Zhang, Xing-Gang Wang, Xiang Bai, Wen-Qing Cheng, and Wen-Yu Liu. Yolop: You only look once for panoptic driving perception. *Machine Intelligence Research*, 19(6):550–562, 2022.
- [84] Tianze Xia, Yongkang Li, Lijun Zhou, Jingfeng Yao, Kaixin Xiong, Haiyang Sun, Bing Wang, Kun Ma, Guang Chen, Hangjun Ye, et al. Drivelaw: Unifying planning and video generation in a latent driving world. In *CVPR*, 2026.
- [85] Pengchuan Xiao, Zhenlei Shao, Steven Hao, Zishuo Zhang, Xiaolin Chai, Judy Jiao, Zesong Li, Jian Wu, Kai Sun, Kun Jiang, Yunlong Wang, and Diange Yang. Pandaset: Advanced sensor suite dataset for autonomous driving. In *ITSC*, 2021.
- [86] Zikang Xiong, Weixin Li, Zhouchonghao Wu, Akshay Rangesh, Saarth Bonde, Grantland Hall, Chen Tang, Yihan Hu, and Wei Zhan. Terratransfer: Learning end-to-end driving policies without expert demonstrations. *arXiv preprint arXiv:2606.17386*, 2026.
- [87] Longfei Xu, Xiaohui Wang, Zehao Huang, Han Li, Ya Yang, Naiyan Wang, and Si Liu. Geometry-grounded unified 3d perception for autonomous driving. *arXiv preprint arXiv:2608.13147*, 2026.

- [88] Honghui Yang, Sha Zhang, Di Huang, Xiaoyang Wu, and Haoyi Zhu. Unipad: A universal pre-training paradigm for autonomous driving. In *CVPR*, 2024. doi: 10.1109/CVPR52733.2024.01443.
- [89] Jiazhi Yang, Shenyuan Gao, Yihang Qiu, Li Chen, Tianyu Li, Bo Dai, Kashyap Chitta, Penghao Wu, Jia Zeng, Ping Luo, Jun Zhang, Andreas Geiger, Yu Qiao, and Hongyang Li. Generalized predictive model for autonomous driving. In *CVPR*, 2024.
- [90] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. In *NeurIPS*, volume 37, 2024. doi: 10.52202/079017-0688.
- [91] Zetong Yang, Li Chen, Yanan Sun, and Hongyang Li. Visual point cloud forecasting enables scalable autonomous driving. In *CVPR*, 2024. doi: 10.1109/CVPR52733.2024.01390.
- [92] Zhenjie Yang, Xiaosong Jia, Qifeng Li, Xue Yang, Maoqing Yao, and Junchi Yan. Raw2drive: Reinforcement learning with aligned world models for end-to-end autonomous driving (in carla v2). In *NeurIPS*, 2025.
- [93] Wenhao Yao, Zhenxin Li, Shiyi Lan, Zi Wang, Xinglong Sun, Jose M Alvarez, and Zuxuan Wu. Drivesuprim: Towards precise trajectory selection for end-to-end planning. In *AAAI*, 2026.
- [94] Yuan Yin, Elias Ramzi, Marc Lafon, Valentin Charraut, Victor Bares, Yihong Xu, ̃ Zablocki, Alexandre Boulch, Thibault Buhet, Andrei Bursuc, et al. Pictura: Perspective-view self-play at scale for driving. *arXiv preprint arXiv:2607.26005*, 2026.
- [95] Chengran Yuan, Zhanqi Zhang, Jiawei Sun, Shuo Sun, Zefan Huang, Christina Dao Wen Lee, Dongen Li, Yuhang Han, Anthony Wong, Keng Peng Tee, and Marcelo H. Ang. Drama: An efficient end-to-end motion planner for autonomous driving with mamba. *arXiv preprint arXiv:2408.03601*, 2024.
- [96] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. *ICCV*, 2023. doi: 10.1109/ICCV51070.2023.01100.
- [97] Dongkun Zhang, Jiaming Liang, Ke Guo, Sha Lu, Qi Wang, Rong Xiong, Zhenwei Miao, and Yue Wang. CarPlanner: Consistent auto-regressive trajectory planning for large-scale reinforcement learning in autonomous driving. In *CVPR*, 2025.
- [98] Kaiwen Zhang, Zhenyu Tang, Xiaotao Hu, Xingang Pan, Xiaoyang Guo, Yuan Liu, Jingwei Huang, Li Yuan, Qian Zhang, Xiao-Xiao Long, et al. Epona: Autoregressive diffusion world model for autonomous driving. In *ICCV*, 2025.
- [99] Ruiqi Zhang, Jing Hou, Florian Walter, Shangding Gu, Jiayi Guan, Florian Röhrbein, Yali Du, Panpan Cai, Guang Chen, and Alois Knoll. Multi-agent reinforcement learning for autonomous driving: A survey. *arXiv preprint arXiv:2408.09675*, 2024.
- [100] Zhejun Zhang, Alexander Liniger, Dengxin Dai, Fisher Yu, and Luc Van Gool. End-to-end urban driving by imitating a reinforcement learning coach. In *ICCV*, 2021.
- [101] Yinan Zheng, Ruiming Liang, Kexin ZHENG, Jinliang Zheng, Liyuan Mao, Jianxiong Li, Weihao Gu, Rui Ai, Shengbo Eben Li, Xianyu Zhan, and Jingjing Liu. Diffusion-based planning for autonomous driving with flexible guidance. In *ICLR*, 2025.
- [102] Hang Zhou, Qiang Zhang, Peiran Liu, Yihao Qin, Zhaoxu Yan, and Yiding Ji. Physics-informed diffusion mamba transformer for real-world driving. In *ICRA*, 2026.
- [103] Hongyu Zhou, Longzhong Lin, Jiabao Wang, Yichong Lu, Dongfeng Bai, Bingbing Liu, Yue Wang, Andreas Geiger, and Yiyi Liao. Hugsim: A real-time, photo-realistic and closed-loop simulator for autonomous driving. *arXiv preprint arXiv:2412.01718*, 2024.
- [104] Xin Zhou, Zongchuang Zhao, Zhibo Yang, Mingsheng Li, Humen Zhong, Shuai Bai, Du Chu, Ruizhe Chen, Zhaohai Li, Jun Tang, Qiuyue Wang, Mingkun Yang, Jiazhao Zhang, Dayiheng Liu, Dingkan Liang, and Xiang Bai. Qwen-drive-1.0: An initial step towards a vision-language foundation model for autonomous driving, 2026.
- [105] Zewei Zhou, Tianhui Cai, Seth Z. Zhao, Yun Zhang, Zhiyu Huang, Bolei Zhou, and Jiaqi Ma. Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. In *NeurIPS*, 2025.

A DriveRL Supplementary Material

A.1 Simulation and Training Infrastructure

A.1.1 Distributed PPO Configuration

Table A1 lists the data, rollout, and PPO settings of the DriveRL teacher. Each rank rolls out 2,048 worlds for 110 steps, giving 225,280 transitions per update, which are split into 11 minibatches of 20,480 for each of the four PPO epochs.

Table A1 Training and rollout configuration. Runtime and optimization settings for DriveRL teacher.

Setting	Value	Setting	Value
World initialization	11 frames / 2 s	Rollout horizon	110 steps at 5 Hz
Worlds per rank	2,048	Traffic quota / rank	1,024 reactive + 1,024 replay
Distributed scale	12 nodes / 96 GPUs	Global rollout batch	196,608 worlds
Nominal transitions / rank	225,280	Optimizer minibatch / rank	20,480
Minibatches / epoch	11	Optimization epochs	4
Optimizer steps / update	44	Training updates	2,400
Training time	approximately 21 h	Optimizer	Muon
Discount γ	0.99	GAE λ	0.95
Policy / value clip	0.2 / 0.2	Entropy coefficient	0.01
Value coefficient	0.5	Gradient clipping	0.5
Advantage normalization	Valid current-rank minibatch	Return normalization	disabled
Initial learning rate	3×10^{-4}	LR schedule	cosine, no warmup

A.1.2 Mixed-Agent Environment and Traffic Providers

The mixed-agent simulator assigns background actors to behavior providers through a common physical state–action interface. The reported training setup uses log replay, IDM, and front-vehicle braking; the same interface also supports policy-controlled actors for self-play.

IDM. The IDM implementation uses an approximately 100 m lane-center search range, a 5 m map-matching threshold, a 1 m minimum gap, a 1.5 s headway, and 0.1 s integration.

Front-vehicle braking. To expose the ego to sudden deceleration of a lead vehicle, which is rare in the logs, the simulator can override the behavior of a background vehicle with an emergency-braking provider. At world initialization, each vehicle already present in the scene and located within a 3 m lateral corridor ahead of the ego is marked as a braking candidate with probability 3%. At every 5 Hz step, each candidate triggers braking with probability 5%. Once triggered, the vehicle decelerates at a rate capped at -10 m/s^2 until it comes to a stop, after which it returns to its default provider.

A.2 Structured Observation and Goal Conditioning

The source loader stores at most 128 actors. The policy filters actors within a 200 m radius, orders them by distance, and retains at most 96 agent tokens with ego fixed in slot 0; 32 slots each are reserved for vehicles and pedestrians. Each actor history contains five frames total, ordered from $t - 0.8 \text{ s}$ to t at 0.2 s intervals. The local map window covers 100 m behind, 200 m ahead, and $\pm 200 \text{ m}$ laterally. Table A2 summarizes the resulting structured Teacher inputs.

Table A2 Structured Teacher input schema. Listed capacities are maximum counts after policy filtering.

Input	Per-item shape	History or capacity	Frame
Goal anchors	2-D point	2 anchors per policy input	Ego
Ego state	9-D	Current frame	Ego
Actor history	15-D / frame	5 frames; ≤ 96 actors including ego	Ego
Vector-map segment	10-D / token	≤ 256 tokens, including ≤ 64 lane centers	Ego
Traffic-light state	4-D / segment	Attached to the corresponding map segment	Map segment

For one policy observation, the goal tensor contains two 2-D points, i.e., $G_t \in \mathbb{R}^{2 \times 2}$; batch and actor dimensions are added only for storage and export. The map loader exposes at most 1,024 lane-center candidates and 1,024 other segment candidates. It first retains at most 64 lane centers, merges them with the remaining segments, and then applies the common spatial and distance filter to obtain at most 256 map tokens. Each raw map token contains two endpoints (4-D), type (1-D), speed limit (1-D), and traffic-light state (4-D). Segments without an associated signal use a zero traffic-light vector. An MLP with endpoint positional encoding maps each 10-D token to a 256-D embedding. Route intent is extracted into goal anchors without a separate route token.

Goals condition the policy. Their construction differs between training and deployment, as summarized in Table A3. Although the goal sources and update schedules differ, both protocols use the same two-anchor, permutation-invariant interface; Section A.5.4 studies sensitivity to the resulting goal layout.

Table A3 Training and deployment goal protocols. Goal sources, update schedules, and slot semantics used by the Teacher.

Aspect	PPO training	Formal nuPlan deployment
Source	Future-log positions	Current ego state + route polyline
Update	Once at reset; fixed per episode	Every planner step
Slots	One or two; duplicate a single goal	Two; near and far route goals
Snapping	50% logged / 50% lane projection	Causal route polyline
Order semantics	Shared encoder + mean pool	Same unordered-set semantics

Training candidates are sampled without replacement with weight $p(\tau) \propto (\tau + 1)$ and then sorted by time. When lane snapping is selected, the sampler considers the four nearest lane-center projections within 20 m and assigns weight $[\max(1 - d/20, 0)]^2$; it keeps the original goal if no valid lane is available. At deployment, ego is projected onto the route polyline and the look-ahead interval is

$$L = \min(L_{\text{remain}}, 12 \max(\|v_t\|, 5)).$$

The two goals lie at its midpoint and endpoint. Exchanging the goal slots does not change their mean-pooled embedding, so they represent an unordered spatial set rather than a waypoint sequence.

A.3 Policy, Action Space, and Vehicle Dynamics

The action head outputs $[\alpha_j, \alpha_\delta, \beta_j, \beta_\delta]$ and applies $\text{softplus}(x) + 1$, ensuring $\alpha, \beta > 1$. The normalized jerk and steering-rate dimensions are conditionally independent given the observation and goal. PPO rollouts sample from both Beta distributions; formal nuPlan evaluation executes their per-dimension analytic mode $(\alpha - 1)/(\alpha + \beta - 2)$. Table A4 lists the physical interface and dynamics constants.

Table A4 Action and vehicle-dynamics parameters. Physical command ranges and constants used by the kinematic bicycle model.

Parameter	Value / range	Unit
Longitudinal jerk	$[-8.00, 5.00]$	m/s^3
Tire steering rate	$[-0.80, 0.80]$	rad/s
Positive jerk cap ($a \geq 0$)	1.00	m/s^3
Wheelbase	3.09	m
Rear axle to center	1.46	m
Maximum acceleration	4.00	m/s^2
Maximum steering angle	$\pi/3$	rad
Maximum steering rate	0.80	rad/s
Acceleration time constant	0.20	s
Steering time constant	0.05	s

An affine transform maps normalized actions into the ranges above. The teacher can export α , β , mean, mode, and sampled actions for student distillation. Exporting both concentration parameters preserves more information about the policy distribution than exporting only the mean, mode, or sampled action.

A.4 Reward, Value Decomposition, and PPO Objective

The reported teacher uses the scalar reward

$$r_t = h_t + (1 - d_t) \left(g_t + \frac{1}{110} \prod_{k \in \mathcal{K}} q_{t,k} \right), \quad \mathcal{K} = \{\text{cross-lane, centerline, curb, comfort, TTC, overspeed}\}. \quad (7)$$

Here, d_t is one when a hard event terminates the rollout. The reward terms are configured as follows:

- **Hard:** newly occurring polygon collisions receive a penalty of -1.0 ; off-road events terminate the rollout and use a speed-dependent penalty with base value -2.0 .
- **Goal:** first arrival within 1.5 m yields a one-time reward of 0.5. Goal-distance shaping and survival rewards are disabled, and goal arrival does not terminate the rollout.
- **Soft:** each score $q_{t,k} \in [0, 1]$ is produced by its corresponding driving-quality calculator. Their product is normalized by the 110-step rollout horizon.

The multiplicative soft term requires the six driving-quality criteria to be jointly satisfactory, preventing strong performance on one criterion from compensating for a serious deficiency in another. The resulting scalar return, however, combines sparse hard events, sparse goal arrivals, and a dense joint soft signal in a single regression target. We therefore use an eight-channel critic,

$$\mathcal{C} = \{\text{hard, goal}\} \cup \mathcal{K}, \quad \mathbf{V}(s_t) = [V_c(s_t)]_{c \in \mathcal{C}} \in \mathbb{R}^8. \quad (8)$$

The hard and goal channels retain their reward semantics. The six soft channels do not correspond to separately defined rewards. They partition the single multiplicative soft reward according to a fixed deficit-based rule, solely to provide structured critic targets while preserving the scalar reward. Specifically, let

$$f_t = (1 - d_t) \frac{1}{110} \prod_{j \in \mathcal{K}} q_{t,j}, \quad b_{t,k} = \max(1 - q_{t,k}, 0), \quad B_t = \sum_{j \in \mathcal{K}} b_{t,j},$$

$$w_{t,k} = \begin{cases} \frac{b_{t,k}}{B_t}, & B_t > \varepsilon_b, \\ \frac{1}{|\mathcal{K}|}, & B_t \leq \varepsilon_b, \end{cases} \quad r_{t,c} = \begin{cases} h_t, & c = \text{hard}, \\ (1 - d_t)g_t, & c = \text{goal}, \\ w_{t,k}f_t, & c = k \in \mathcal{K}. \end{cases} \quad (9)$$

Here, $\varepsilon_b = 10^{-8}$. The uniform fallback avoids an undefined allocation when all six soft scores are perfect. Since $\sum_{k \in \mathcal{K}} w_{t,k} = 1$, the eight component rewards exactly conserve the scalar reward, i.e., $\sum_{c \in \mathcal{C}} r_{t,c} = r_t$.

Generalized advantage estimation is applied channel-wise using the same discount, trace parameter, and terminal boundary:

$$\begin{aligned}\delta_{t,c} &= r_{t,c} + \gamma n_t V_c(s_{t+1}) - V_c(s_t), \\ A_{t,c} &= \delta_{t,c} + \gamma \lambda n_t A_{t+1,c}, \quad c \in \mathcal{C}, \quad n_t = 1 - d_t.\end{aligned}\tag{10}$$

The final rollout state supplies the usual bootstrap value. We use $\gamma = 0.99$ and $\lambda = 0.95$, and train each value channel toward $\hat{R}_{t,c} = A_{t,c} + V_c(s_t)$.

The actor uses the summed advantage $A_t = \sum_{c \in \mathcal{C}} A_{t,c}$, normalized over valid samples within each minibatch, in the standard clipped PPO surrogate. Because the component rewards sum to r_t and all channels share the same (γ, λ, n_t) , linearity of GAE makes this summed advantage equivalent, up to floating-point error, to scalar GAE applied to r_t and the total value $V_{\text{tot}}(s_t) = \sum_{c \in \mathcal{C}} V_c(s_t)$. For critic optimization, a PPO-style clipped value loss is computed independently for each channel, and the eight losses are summed before applying the value-loss coefficient. The decomposition therefore structures critic supervision and diagnostics while preserving the scalar policy objective in Eq. (7).

Traffic-light violations are reported separately as a diagnostic metric; they are not part of the reported teacher’s scalar reward. PPO and distributed settings appear in Table A1.

A.5 Additional DriveRL Analyses

Unless otherwise stated, analyses in this section use the reported DriveRL checkpoint. Some controlled ablations were conducted in an earlier 32-GPU training setup and are used only for within-table comparisons; their absolute scores should not be compared directly with the reported teacher.

A.5.1 Detailed nuPlan Metric Breakdown

Table A5 reports the component metrics for all six nuPlan evaluation settings.

Table A5 Detailed nuPlan metrics for DriveRL. Results across six evaluation settings; higher is better for every column.

Split / mode	Score	Collision	Drivable	Direction	Making progress	TTC	Progress	Speed	Comfort
Val14 NR	95.16	98.75	100.00	99.60	98.93	95.97	96.30	98.80	98.84
Val14 R	94.25	98.57	100.00	99.24	98.75	95.80	93.83	99.21	98.66
Test14-hard NR	89.97	98.90	98.90	98.16	98.16	93.38	91.74	97.67	95.96
Test14-hard R	89.18	97.43	99.26	98.53	96.69	94.49	88.73	98.69	96.32
Test14-random NR	94.50	98.08	99.23	99.81	100.00	95.40	95.48	98.33	98.85
Test14-random R	95.00	98.85	99.62	100.00	99.62	96.55	93.34	98.85	98.85

A.5.2 Self-Play Study

The DriveRL-SelfPlay assigns the same policy to the ego vehicle and up to ten vehicle NPCs. Four IDM-controlled vehicles are first included in the candidate set, and the remaining slots are filled with the most relevant policy-controlled vehicles. Table A6 summarizes the configuration differences between DriveRL-SelfPlay and DriveRL.

Table A6 Configuration differences between DriveRL and DriveRL-SelfPlay. The two policies use the same mixed-agent simulator and PPO objective.

Setting	Single-Ego Teacher	Self-Play Teacher
Policy-controlled actors	Ego only	Ego + ≤ 10 vehicle NPCs
Traffic composition	Log/IDM mixed50	Policy NPCs + log/IDM mixed50
Training samples	922,703	909,645 (subset of Single-Ego data)
Worlds per rank	2,048	512
Global rollout worlds	196,608	49,152

The following comparison reports the Single-Ego baseline and the Self-Play variants. Among the latter, disabling history dropout, dynamics noise, and front-vehicle braking gives the highest mean score. Its mean performance is comparable to that of the Single-Ego baseline, demonstrating the feasibility of the one-policy, per-actor formulation but not a consistent performance advantage from Self-Play.

Table A7 Self-Play Comparison on the Six Closed-Loop Evaluations. All Self-Play variants are trained for 4,800 updates; the Single-Ego baseline uses its reported 2,400-update checkpoint. The best checkpoint is reported for each run, and Mean is the unweighted average over the six evaluations.

Model	Val14 $\uparrow$		Test14-hard $\uparrow$		Test14-random $\uparrow$		Mean $\uparrow$
	NR	R	NR	R	NR	R	
DriveRL	95.16	94.25	89.97	89.18	94.50	95.00	93.01
Self-Play, 3-perturb + augmentation	95.08	94.48	88.30	87.27	94.60	94.68	92.40
Self-Play, 3-perturb	95.15	93.74	88.90	87.47	93.70	94.77	92.29
Self-Play, augmentation	94.91	94.38	88.56	87.92	93.43	94.22	92.24
Self-Play	95.09	93.92	90.04	88.96	95.04	95.63	93.11

A.5.3 Traffic-Light Input and Reward

We study traffic-light conditioning using a separate controlled training setup; absolute scores in this subsection are therefore not directly comparable to those of the reported teacher. We first test the effect of adding traffic-light state to the policy. Each map segment carries a four-dimensional unknown/red/yellow/green feature. Table A8 compares policies with and without this feature.

Table A8 Traffic-Light Input Ablation. The red-light detector is diagnostic only in both settings.

TL input	Val14 $\uparrow$		Test14-hard $\uparrow$		Test14-random $\uparrow$	
	NR	R	NR	R	NR	R
Off	93.35	93.19	86.49	85.41	93.48	92.24
On	94.76	93.62	88.35	85.70	95.21	93.89
Difference	+1.41	+0.43	+1.86	+0.29	+1.73	+1.65

Exposing traffic-light state improves scores in all six evaluation settings.

Within the same setup, we next add a red-light training objective to a policy with traffic-light input. The table reports the policy-only red-light violation rate, which counts violations in scenes where the logged human remains compliant, following NAVSIM’s human-reference filtering principle [18].

Table A9 Traffic-Light Reward and Violation-Termination Ablation. All runs use 32 GPUs; “pp” denotes percentage points.

Objective	Policy-only violation	Val14 $\uparrow$		Test14-hard $\uparrow$		Test14-random $\uparrow$	
		NR	R	NR	R	NR	R
TL input only	3.88%	94.29	93.19	84.85	85.65	93.79	93.09
+ TL reward	2.03%	93.08	91.35	81.71	82.96	92.36	91.79
Difference	−1.85 pp	−1.21	−1.84	−3.14	−2.69	−1.43	−1.30

The traffic-light reward lowers the policy-only violation rate from 3.88% to 2.03%, while all six aggregate scores decrease. This trade-off shows that the aggregate nuPlan score alone does not capture the improvement in red-light compliance, since traffic-light violations are not part of the benchmark score. We therefore report the violation rate alongside the aggregate metrics.

A.5.4 Goal-Anchor Layout

DriveRL always retains two goal input slots. A single-anchor condition duplicates the same goal into both slots and therefore leaves the network architecture unchanged. The formal route-goal interface produces a near anchor g_{near} and a far anchor g_{far} at every planner step.

Fixed-checkpoint sensitivity. We first fix the reported DriveRL checkpoint and vary only the goal layout used for nuPlan evaluation. The default uses the near and far anchors, while the other settings duplicate either anchor into both slots.

Table A10 Goal-Anchor Layout Evaluation for the Fixed Single-Ego Teacher. The highest score in each row is bold.

Split / mode	Default [$g_{\text{near}}, g_{\text{far}}$]	Near [$g_{\text{near}}, g_{\text{near}}$]	Far [$g_{\text{far}}, g_{\text{far}}$]
Val14 NR	95.16	94.97	94.36
Val14 R	94.25	94.00	93.22
Test14-hard NR	89.97	90.74	88.01
Test14-hard R	89.18	88.82	87.13
Test14-random NR	94.50	94.57	93.18
Test14-random R	95.00	95.23	92.84

The default and near-only layouts have similar average performance, while duplicating the far anchor lowers all six scores. This indicates that the near anchor provides important local guidance; the far anchor is most useful when paired with, rather than substituted for, the near anchor.

Training-distribution ablation. We additionally study whether this preference is consistent with the goal distribution used during training, using a separate controlled training setup. Absolute scores in this experiment are therefore not directly comparable to those of the reported teacher. The default distribution samples one or two future-log goals with equal probability and is evaluated with the near–far pair. The other models always sample two goals, retain either the earlier or later one, duplicate it into both slots, and are evaluated with the corresponding near–near or far–far layout.

Table A11 Goal-Anchor Training Distribution Ablation. Models are trained from scratch with 32-GPU Adam, and evaluation layouts are matched to the corresponding training distributions. Mean is the unweighted average over the six evaluations.

Training distribution	Evaluation	Val14 $\uparrow$		Test14-hard $\uparrow$		Test14-random $\uparrow$		Mean $\uparrow$
		NR	R	NR	R	NR	R	
Single / dual (1:1, default)	Near + far	94.76	93.62	88.35	85.70	95.21	93.89	91.92
Earlier of two duplicated	Near + near	94.23	93.35	86.78	87.38	94.73	93.23	91.62
Later of two duplicated	Far + far	93.04	92.43	83.10	82.31	93.27	90.95	89.18

The mixed distribution achieves the highest mean score. With evaluation matched to training, earlier-goal duplication remains close to the default, whereas later-goal duplication performs substantially worse.

B DriveZero Supplementary Material

B.1 Models and Training

Table A12 summarizes the model and training hyperparameters of the camera-only DriveZero-Scale.

Table A12 Model and Training Configuration for DriveZero-Scale.

Hyperparameter	Value
Model Configuration	
Sensors	CAM_F0, CAM_B0, CAM_L0, CAM_R0
Input resolution	960×512 (W×H)
Prediction horizon / frequency	20 steps at 5 Hz
Trajectory proposals	64
Planning representation	256 dimensions
Proposal decoder	4 layers
Register tokens	16 per camera
Visual backbone	DriveVFM ViT-L (frozen)
Backbone adaptation	Q/V LoRA, rank 32
Trainable parameters	18.58 M
Full parameters	338.46 M
Training Configuration	
Training data	100K <i>navtrain</i> + 237K <i>SimScale</i> scenes
GPUs	$16 \times \text{H20}$
Epochs	25
Batch size	256
Optimizer	AdamW
Initial learning rate	2×10^{-4}
Learning-rate schedule	Cosine annealing to zero (no warmup)
Weight decay	0.01
Gradient clipping	Global norm 1.0
Training precision	FP32
Training time	38h
Trajectory supervision	Teacher rollouts; no human trajectories

B.2 Visual Backbone Scaling

We study visual-backbone scaling by comparing DriveVFM ViT-S, ViT-B, and ViT-L. All three backbones are pretrained with the same data sources and identical mixture ratios. For downstream training, all three DriveZero variants use only the NAVSIM *navtrain* split, without any *SimScale* data. The pretrained backbone weights remain frozen, and the same rank-32 Q/V LoRA adapters are fine-tuned for every backbone. All other settings are held fixed. Table A13 reports the 6 PDM components and the aggregate PDMS.

Table A13 Visual-backbone scaling for DriveZero on the NAVSIMv1 navtest benchmark. The backbones use the same data sources and identical mixture ratios. All downstream DriveZero variants use NAVSIM `navtrain` without SimScale and fine-tune rank-32 Q/V LoRA adapters.

Backbone	NC $\uparrow$	DAC $\uparrow$	TTC $\uparrow$	Comfort $\uparrow$	EP $\uparrow$	DDC $\uparrow$	PDMS $\uparrow$
DINOv3 ViT-L	98.80	99.25	95.74	99.94	92.72	95.87	94.55
DriveVFM ViT-S	99.17	99.14	96.29	99.98	91.79	95.97	94.41
DriveVFM ViT-B	98.89	99.24	95.93	99.97	92.44	95.79	94.54
DriveVFM ViT-L	99.00	99.24	95.97	99.99	93.10	95.89	94.83

PDMS increases from 94.41 with ViT-S to 94.54 with ViT-B and 94.83 with ViT-L. The 0.42-point gain from ViT-S to ViT-L shows a positive scaling trend across these backbones. Furthermore, this scaling trend holds even against stronger general-purpose backbones: our largest ViT-L variant still edges out DINOv3 ViT-L (94.55 PDMS) despite the latter’s much larger pretraining scale.

B.3 QK-Clip for Stable Training and FP16 Deployment

Training transformers can cause query–key dot products to grow in some attention heads. The resulting large pre-softmax logits lead to loss spikes and make optimization sensitive to numerical precision. Query–Key Normalization [19, 26] addresses this issue by changing the attention computation itself. DriveVFM instead uses QK-Clip, inspired by Kimi K2 [54]: after an optimizer update, it monitors the maximum pre-softmax logit in each head and rescales the query and key projection weights when the value exceeds a target threshold of 1,000. The query and key receive the same square-root scaling factor, while the value projection and the forward attention formulation remain unchanged. This provides a direct bound on the effective QK scale without introducing an additional normalization operation.

Figure A1 compares optimization with and without QK-Clip. Constraining the query–key scale produces a smoother descent and a lower loss in the latter part of training, supporting its role in stabilizing the training.

The same bound is important at deployment. Converting the backbone from FP32 to FP16 can perturb attention logits and propagate errors through the feature stack. We therefore compare FP32 and FP16 backbone outputs under three training configurations: baseline, with QK normalization, and with QK-Clip. Table A14 reports the maximum and mean absolute deviations together with cosine similarity. The reported maximum QK logit is the largest pre-softmax query–key dot product observed across all attention layers. QK-Norm lowers the maximum QK logit to 5,125 and improves both error metrics (mean abs 0.032, cosine 0.981), but a substantial residual error remains, showing it does not fully resolve the instability. QK-Clip further reduces the mean absolute error to 0.004 and preserves a cosine similarity of 0.999, indicating near-lossless FP16 deployment.

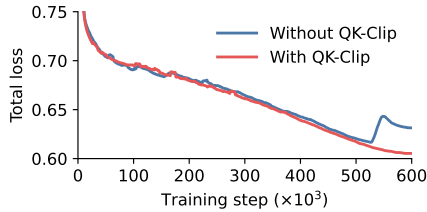


Figure A1 Training loss with and without QK-Clip. QK-Clip stabilizes training, avoiding the loss spikes seen without it.

Method	Max QK Logit	Max Abs	Mean Abs	Cosine
Baseline	308,320	1.243	0.109	0.893
QK-Norm	5,125	0.768	0.032	0.981
QK-Clip	995	0.047	0.004	0.999

Table A14 FP32-to-FP16 feature deviation. QK-Clip yields the lowest deviation, showing that bounding logits also improves FP16 robustness.

B.4 More Visualizations of DriveZero



Figure A2 More Visualization Results of **DriveZero-Scale** vs. **Previous SOTA Model** in NAVSIM navtest [18] Benchmark.